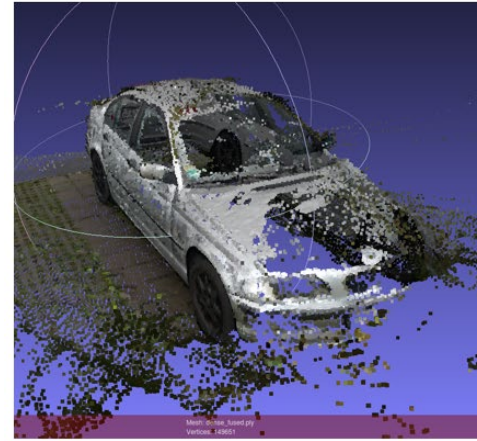
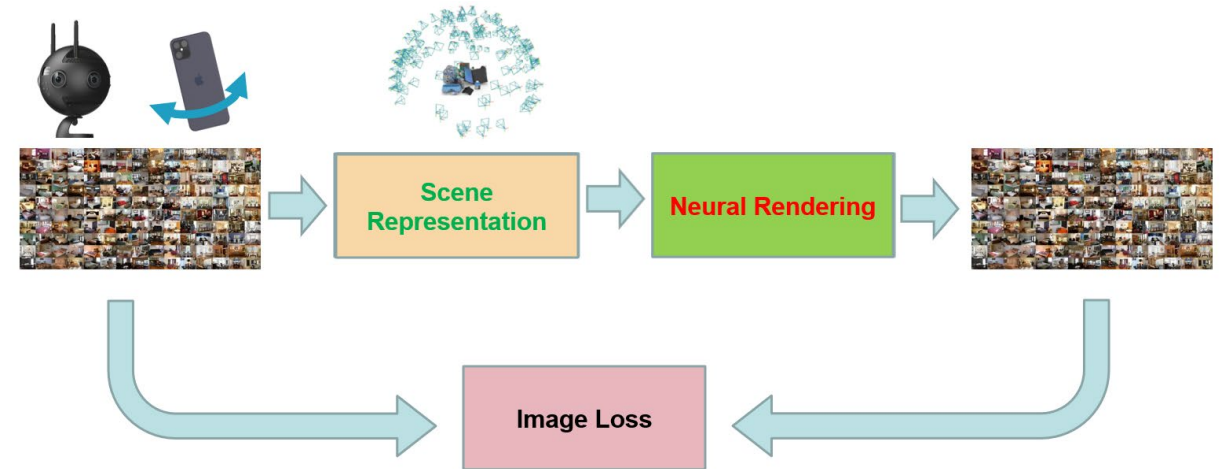


Review of Last Class

1. Challenges in using classical computer graphics pipeline for 3D reconstruction and photorealistic rendering
2. Neural scene representation and neural rendering is the rescue
3. Neural Rendering:
Deep neural networks for **image or video generation** that enable **explicit or implicit control of scene properties**

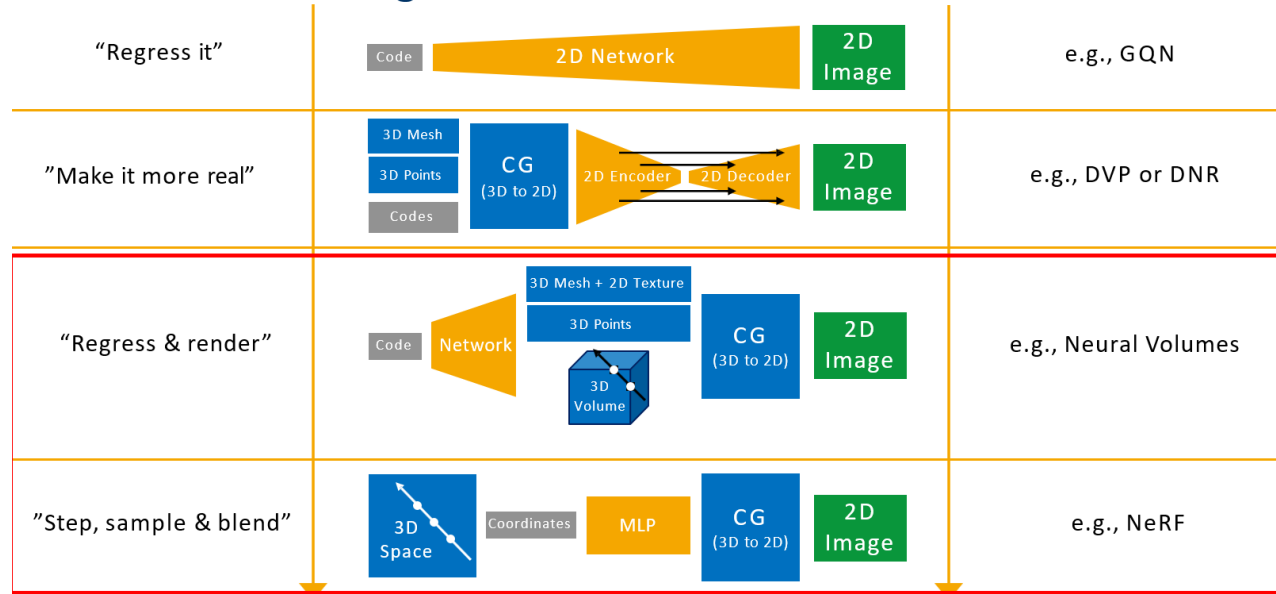


VS



Review of Last Class

Neural Rendering Zoo



4. Different neural rendering methods
- Using neural rendering to learn neural fields.

TODOs After this Class

1. **Paper Selection and Registration:** **[Important! Deadline: Aug 27]**

Please select and register for the two papers you would like to present using the following Excel link:

<https://docs.google.com/spreadsheets/d/1qDT8KQOgKzVdRVaSnWr1UsH5S1WGbp75lvX92QaM124/edit?gid=333550507#gid=333550507>

2. **Presentation Preparation:**

- Please have your slides ready and upload them **at least one hour before the class before your presentation.**
- Please upload your slides to the Google folder: https://drive.google.com/drive/folders/1Y0bF83p6U_SGPmvfi7zcYlAvqaUa6Hp6. This early deadline allows us to reschedule your presentation if an emergency arises.
- For example, if you're presenting on Tuesday, upload your slides by the previous Thursday at 12:45 PM. If presenting on Thursday, upload by Tuesday at 12:45 PM.



3. **Class Participation:**

- Before each class, please read the papers that will be discussed and submit two questions **at least one hour before the class** using the following link:

<https://docs.google.com/forms/d/e/1FAIpQLSfJ-UoJZQOpQS1ePEeezH8-c4zkkUDI1EhNzqXGJ9ExCPYoFg/viewform>

Course Link:

<https://neural-representation-2026.github.io/index.html>



Overview of This Class

0. Neural Fields

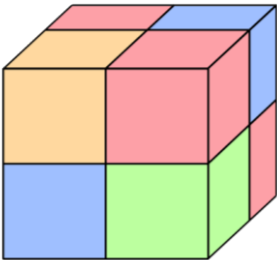
- I. Fundamentals of Classical Rendering Techniques in Computer Graphics
- II. Two pioneering works in Neural Scene Representations and Neural Rendering
 - Scene Representation Networks (SRN)
 - Neural Volumes (before that: Deep Appearance Models)

Neural Scene Representation and Neural Rendering

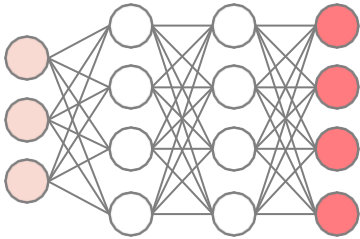
Neural Fields

Scene Representation

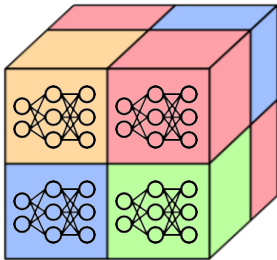
Renderer



Voxelgrids



Implicit Function



Hybrid
Implicit/Explicit

Volumetric

Sphere-Tracing
Volumetric

Volumetric

Pros

Fast rendering

High quality
Compact
Supports *global* priors

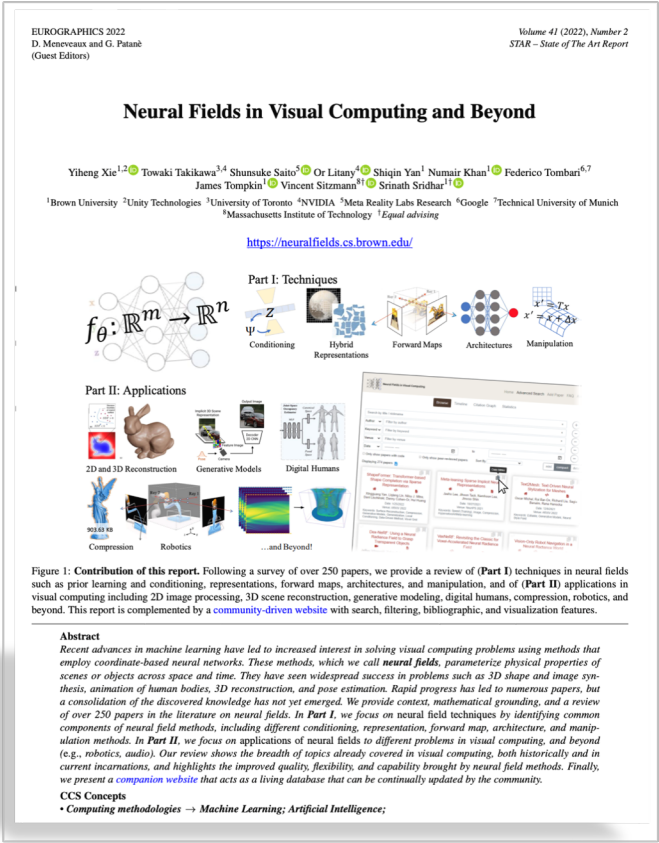
Significant Speedup
Supports *local* priors

Cons

Memory $O(n^3)$
Limited spatial
resolution

Extremely expensive,
slow rendering

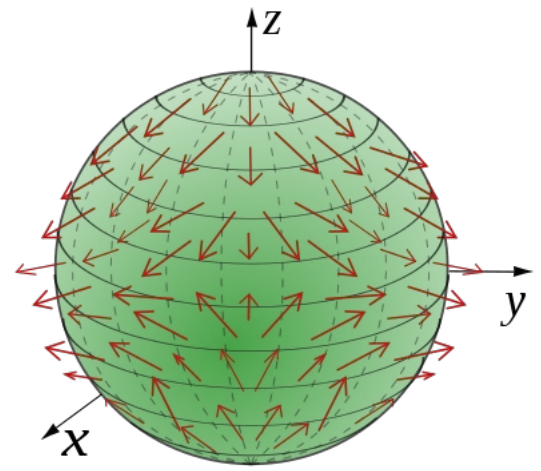
No compact
representation No
global priors



Definition of Fields

A field is a quantity defined for all spatial and / or temporal coordinates.

Examples of Fields

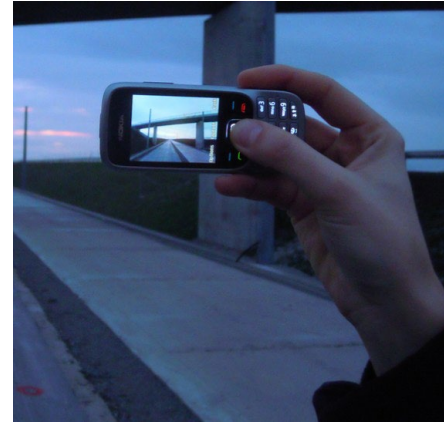


Vector Field

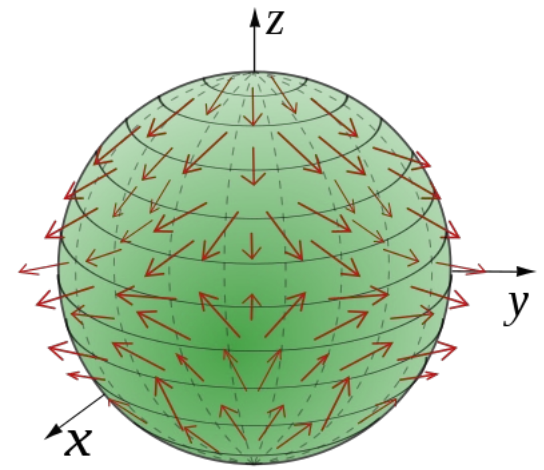


Fields

Examples of Fields

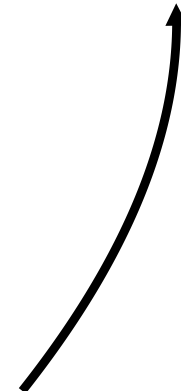
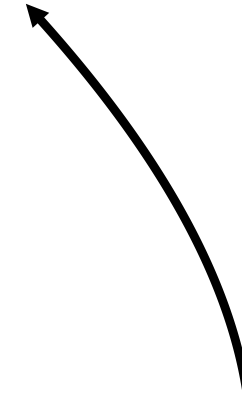


Image

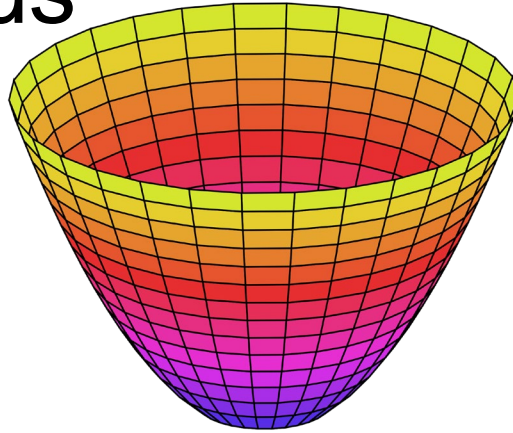


Vector Field

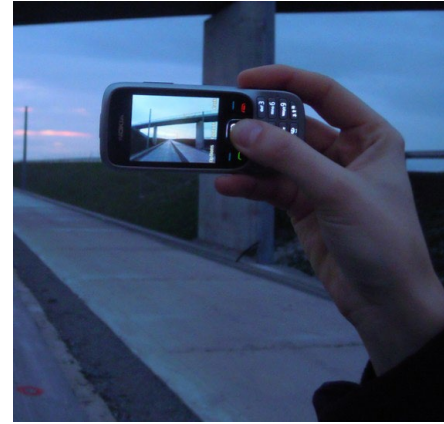
Fields



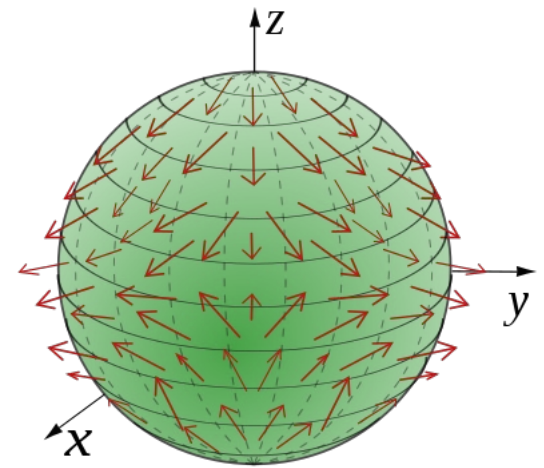
Examples of Fields



3D Parabola
(Explicit Surface)



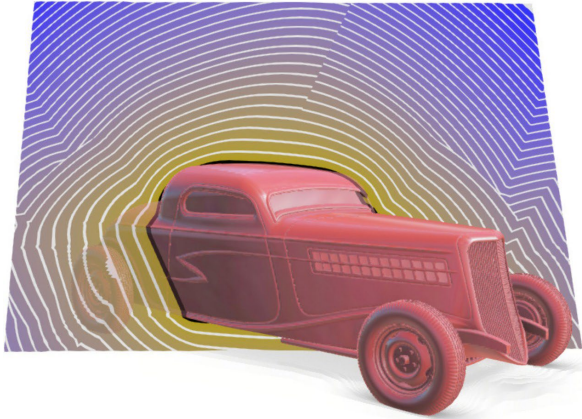
Image



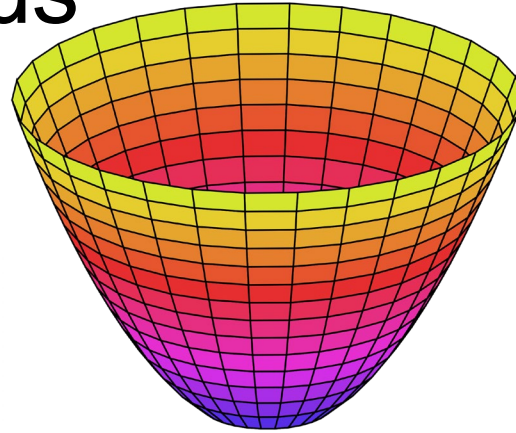
Vector Field

Fields

Examples of Fields



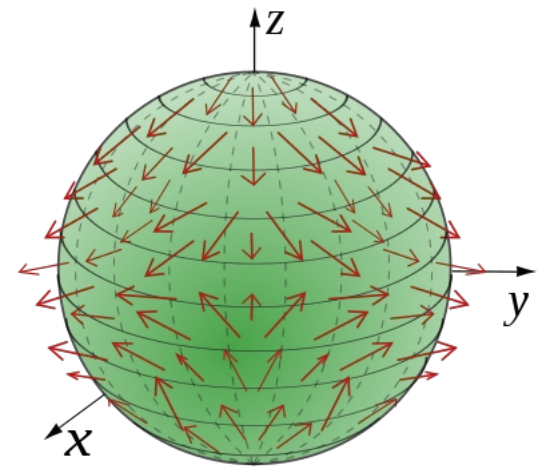
3D Signed Distance Fields
(Implicit Surface)



3D Parabola
(Explicit Surface)

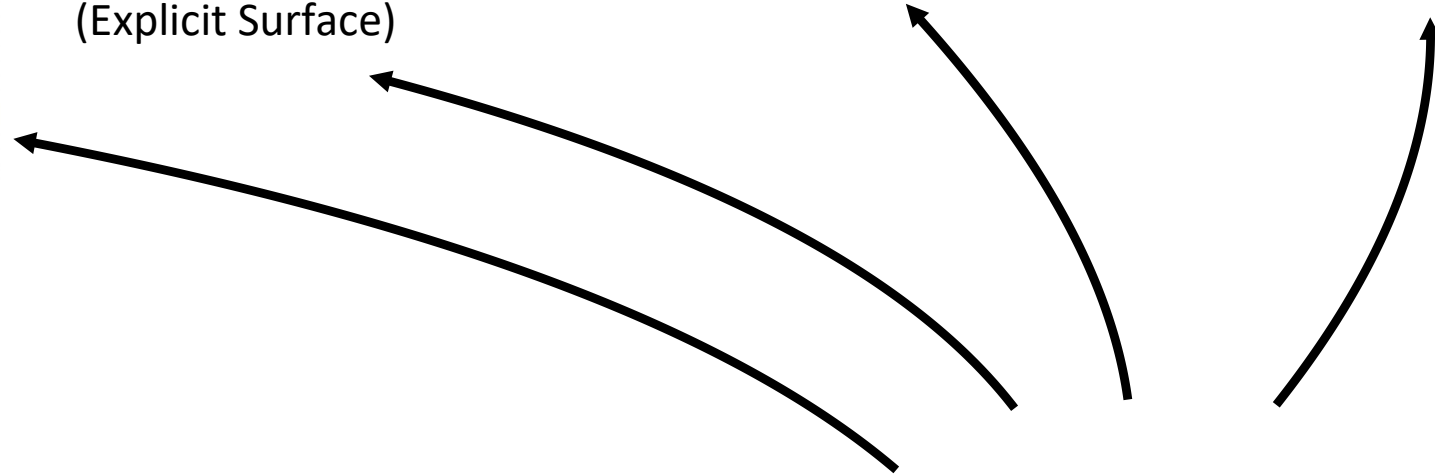


Image

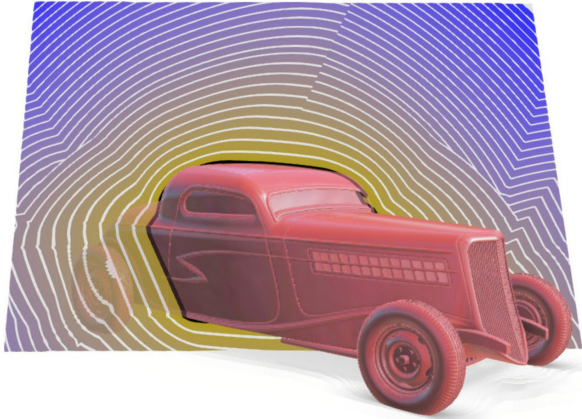


Vector Field

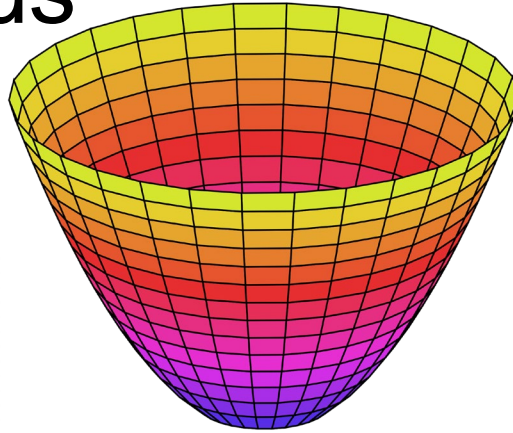
Fields



Examples of Fields



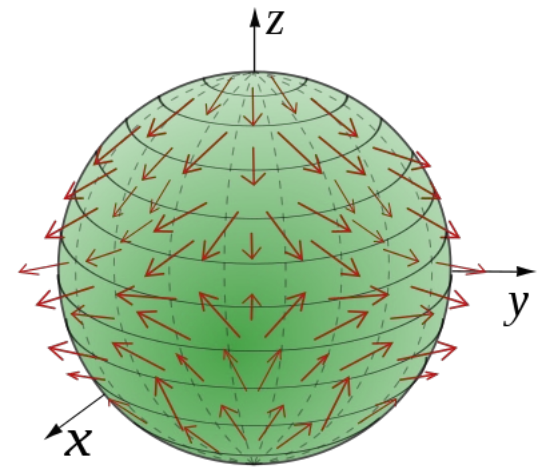
3D Signed Distance Fields
(Implicit Surface)



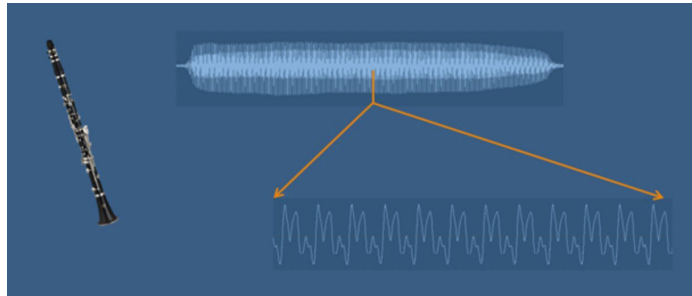
3D Parabola
(Explicit Surface)



Image

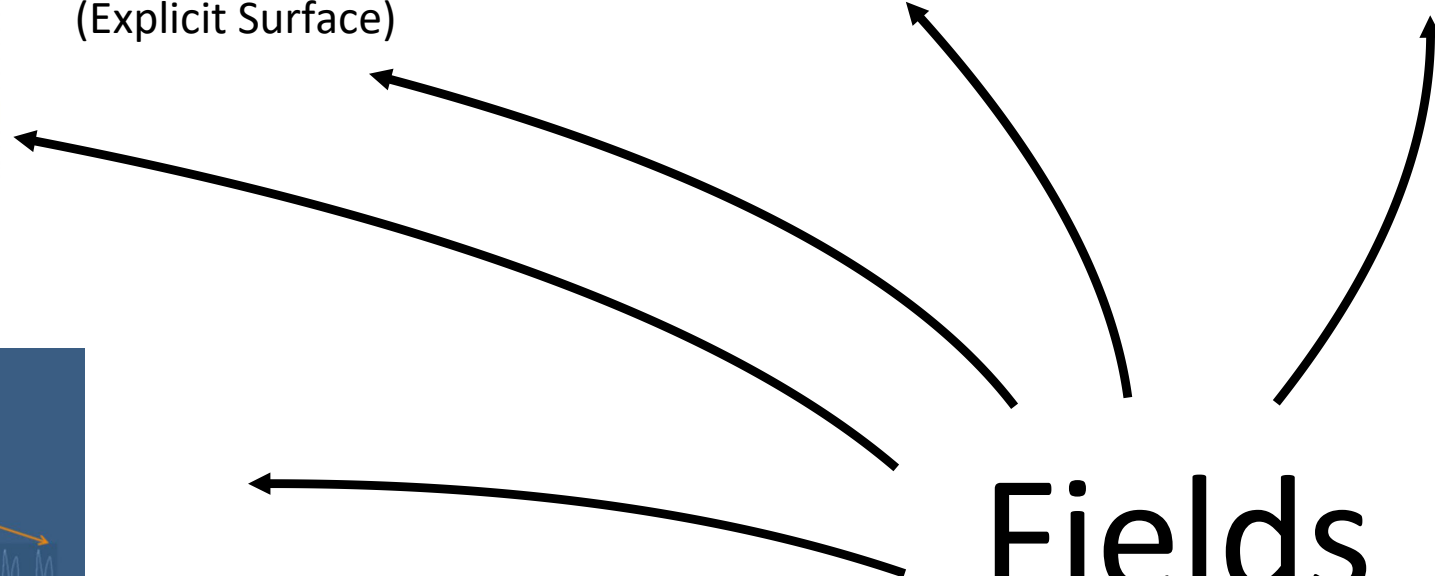


Vector Field



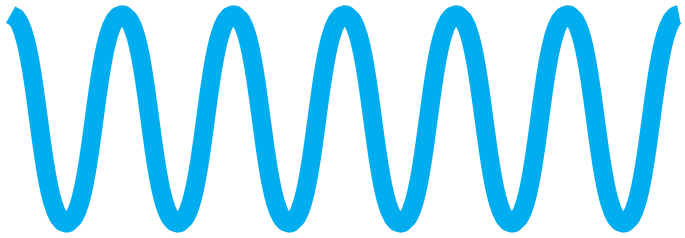
Audio

Fields



What are neural fields?

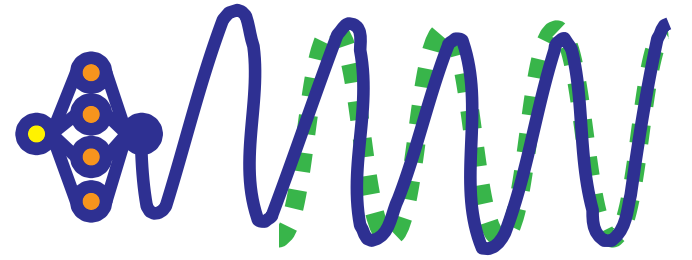
Fields / signals can be represented in many ways.



Continuous

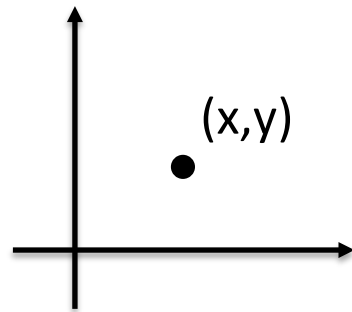


Discrete

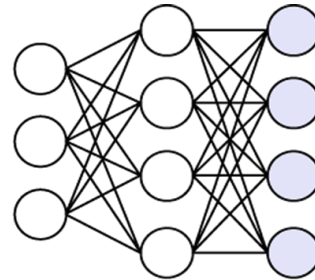


Neural

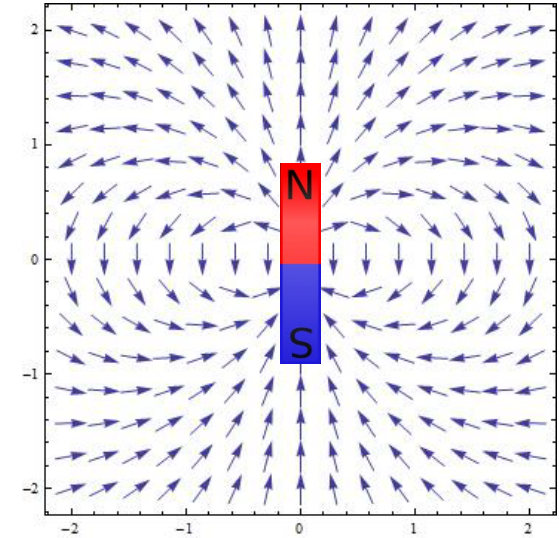
What are neural fields?



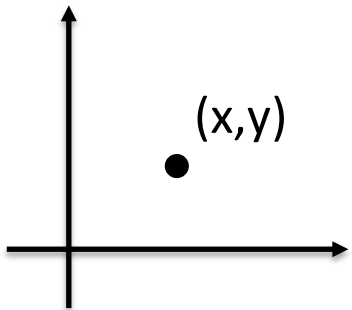
$$\Phi: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$



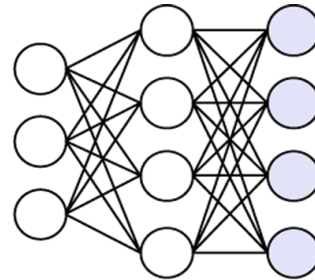
Neural Network (Φ)



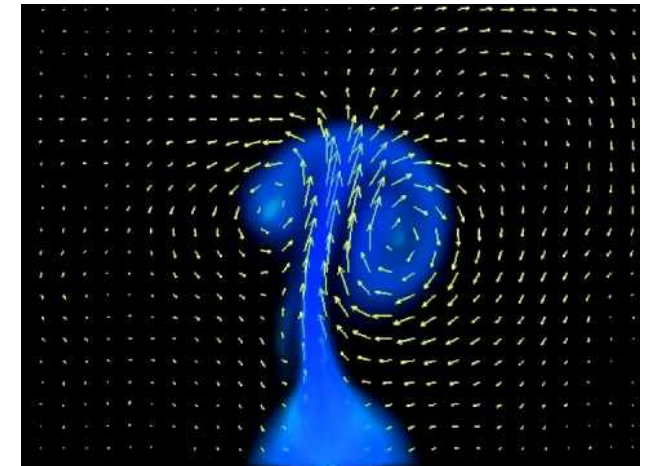
Magnetic Field



$$\Phi: \mathbb{R}^2 \rightarrow \mathbb{R}^2$$



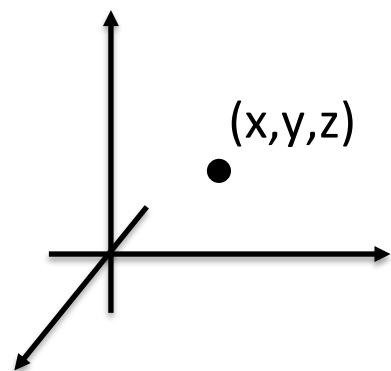
Neural Network (Φ)



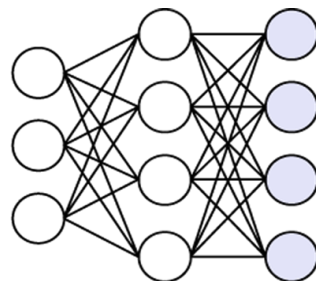
Eulerian Flow Field of a Fluid

[Koldora CC]

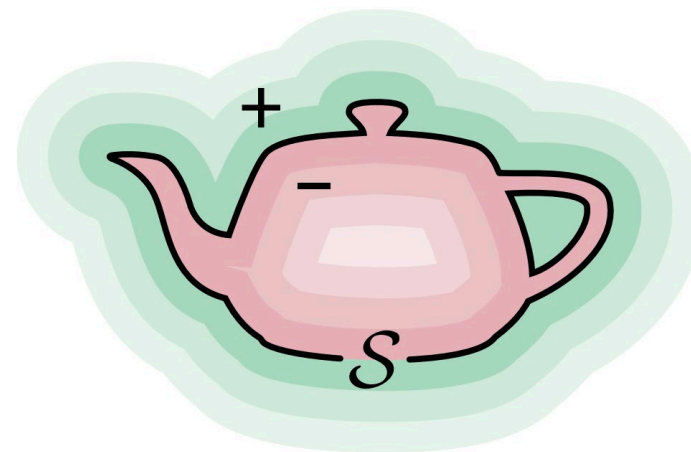
What are neural fields?



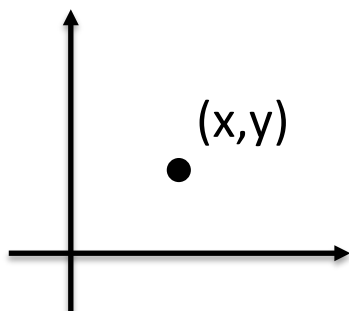
$$\Phi: \mathbb{R}^n \rightarrow \mathbb{R}$$



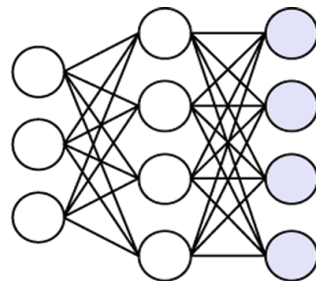
Neural Network (Φ)



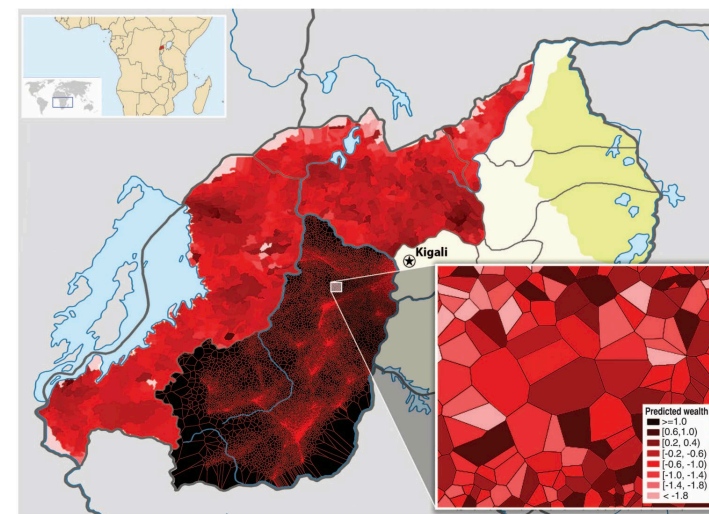
Signed Distance Function (SDF)



$$\Phi: \mathbb{R}^2 \rightarrow \mathbb{R}^n$$



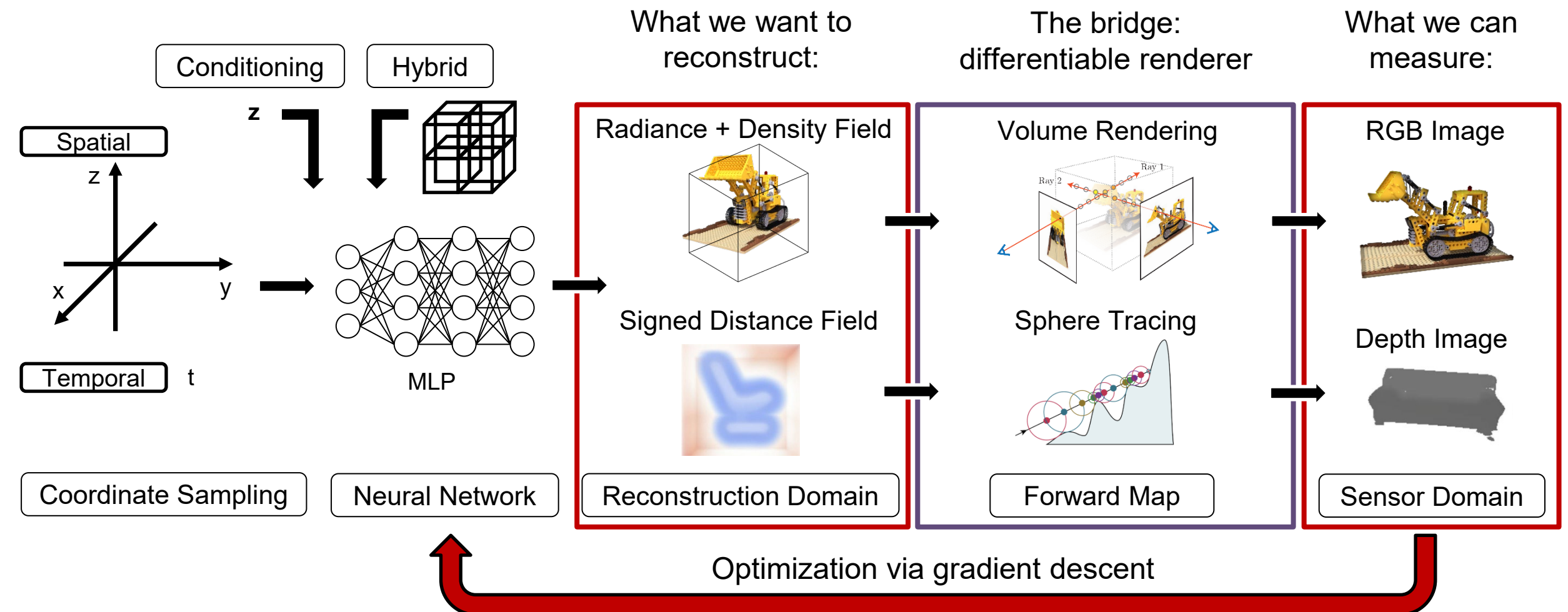
Neural Network (Φ)



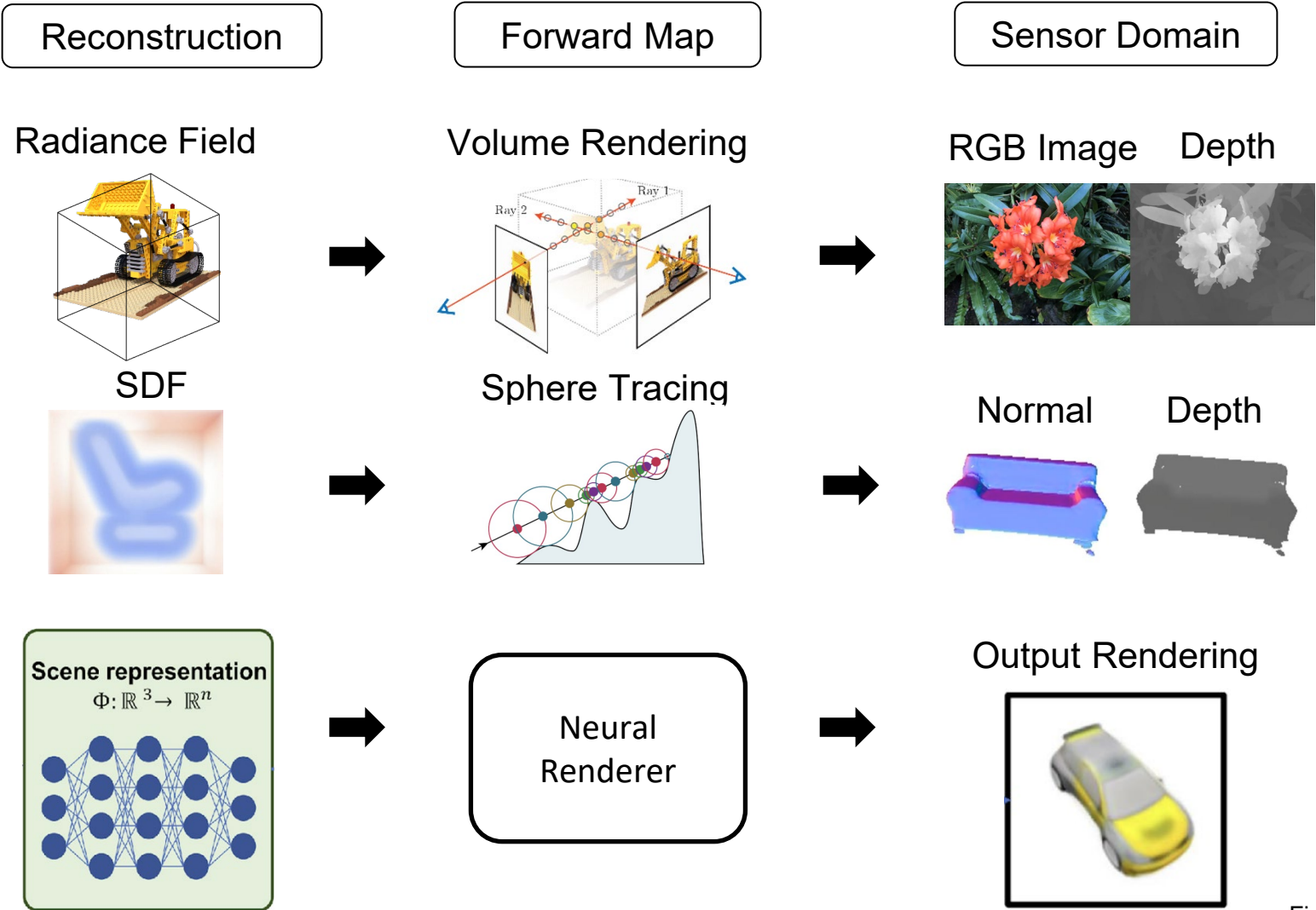
Geospatial Data

[Blumenstock et al. 2015]

Neural Fields General Framework

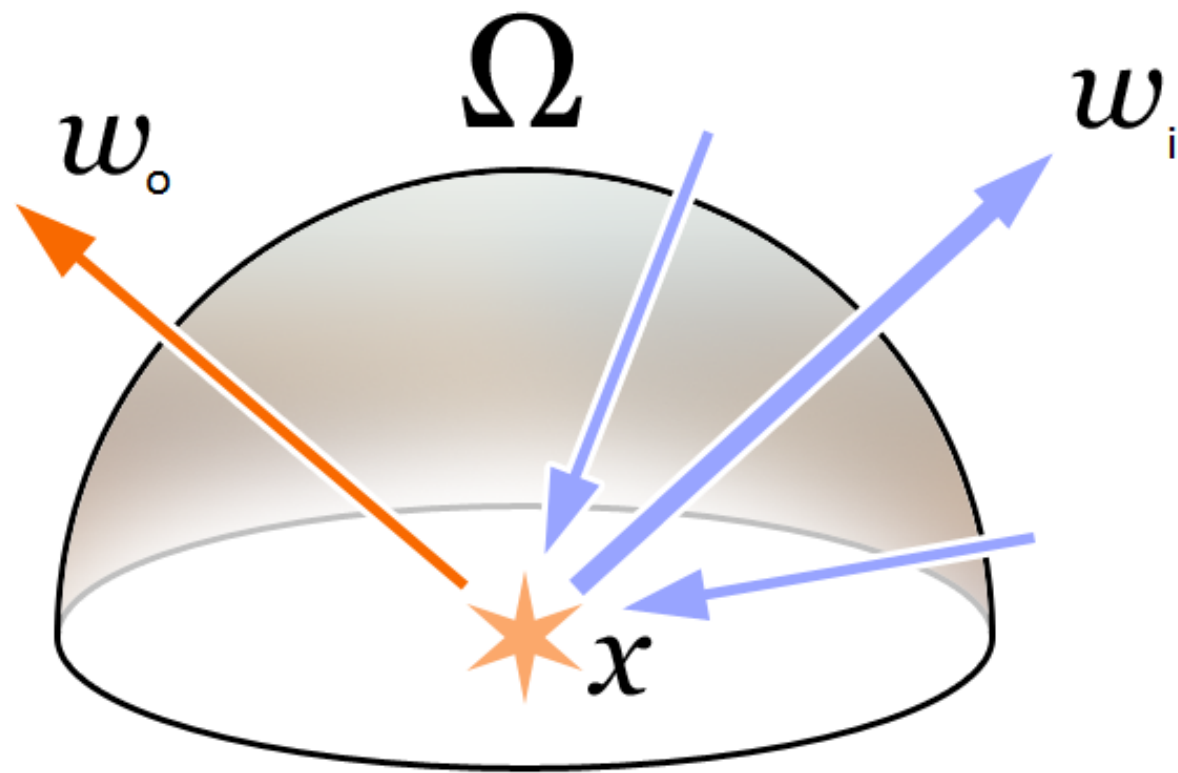


Differentiable Rendering



Figures adapted from:
Mildenhall et al. 2020 (NeRF)
Sitzmann et al. 2019 (SRN)

BRDF Shading



$$L(\mathbf{x}, \vec{\omega}_o) = L_e(\mathbf{x}, \vec{\omega}_o) + \int_{\Omega} f_r(\mathbf{x}, \vec{\omega}_i \rightarrow \vec{\omega}_o) L(\mathbf{x}', \vec{\omega}_i) G(\mathbf{x}, \mathbf{x}') V(\mathbf{x}, \mathbf{x}') d\omega_i$$

Overview of This Class

0. Neural Fields

I. Fundamentals of Classical Rendering Techniques in Computer Graphics

- ### II. Two pioneering works in Neural Scene Representations and Neural Rendering
- Scene Representation Networks (SRN)
 - Neural Volumes (before that: Deep Appearance Models)

Computer Graphics



```
graph LR; A[Geometry Processing] --> B[Rendering]; B --> C[Animation / Simulation];
```

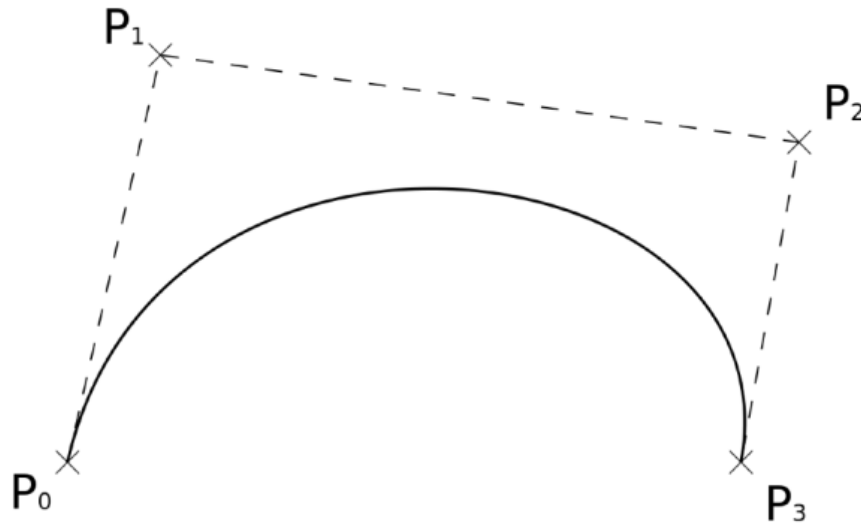
Geometry Processing

Rendering

Animation / Simulation

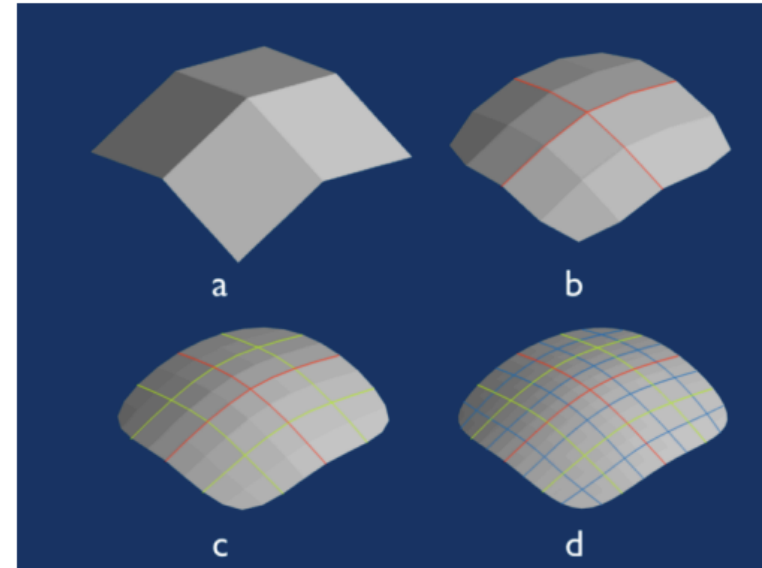
Curves and Meshes

- How to represent geometry in Computer Graphics



Bezier Curve

https://en.wikipedia.org/wiki/Bezier_curve



Catmull-Clark subdivision

https://commons.wikimedia.org/wiki/File:Catmull-Clark_subdivision_of_4_planes.png

Other 3D Representations

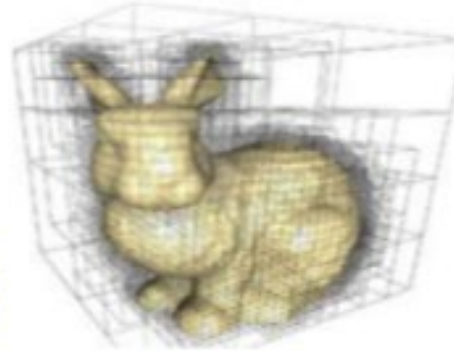
Voxel



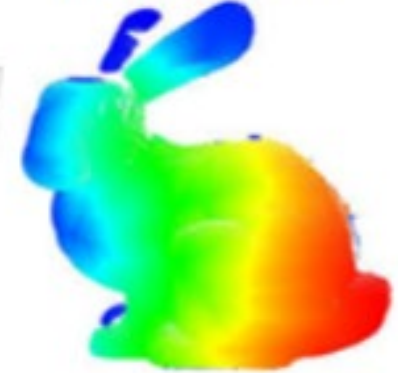
Point cloud



Octree



Descriptor



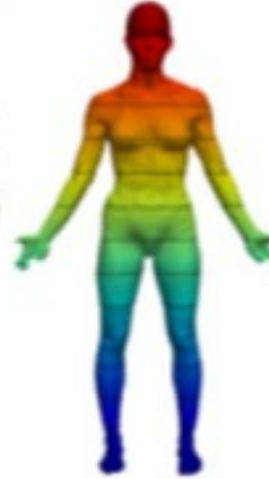
Multi view



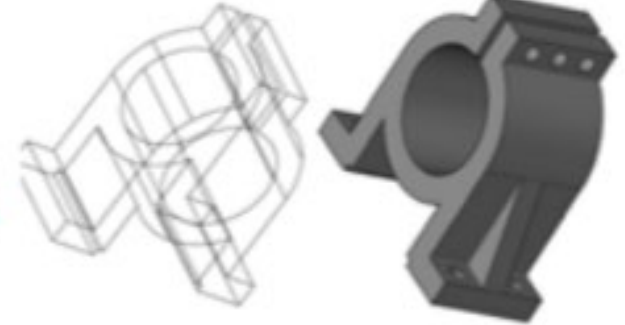
RGB-D



Graph



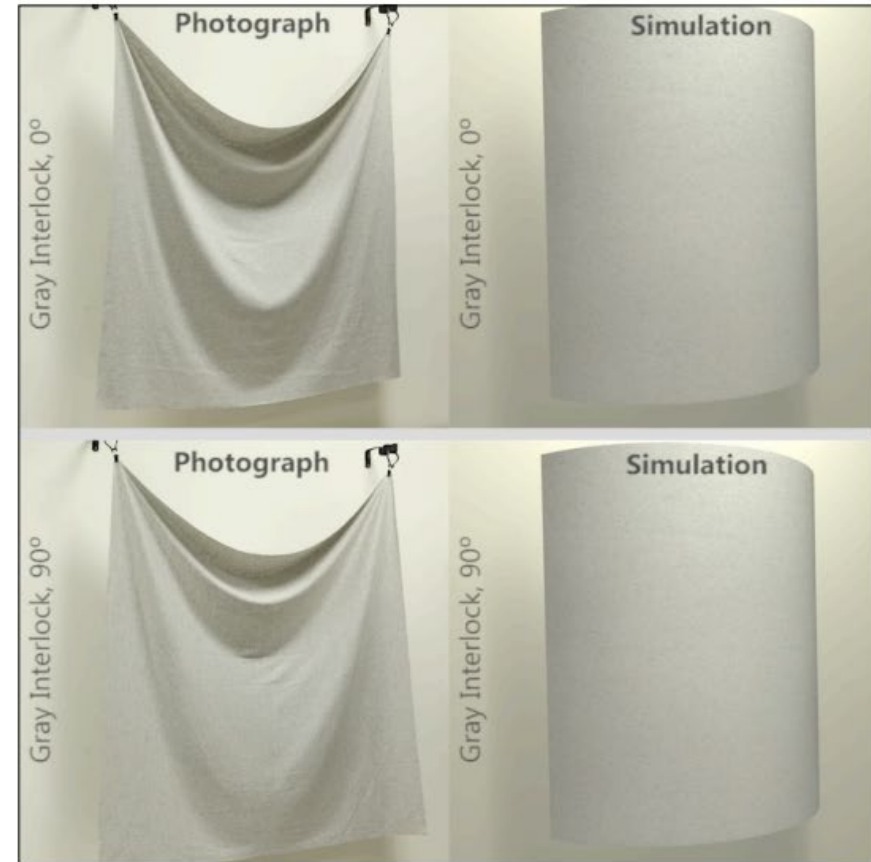
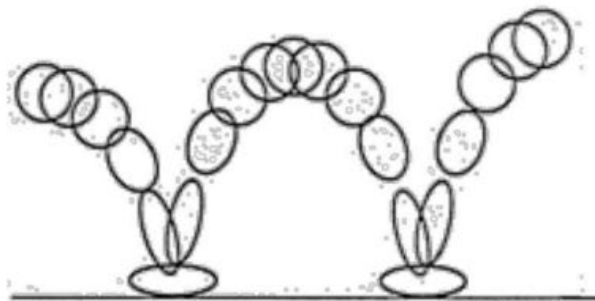
Projections



CAD

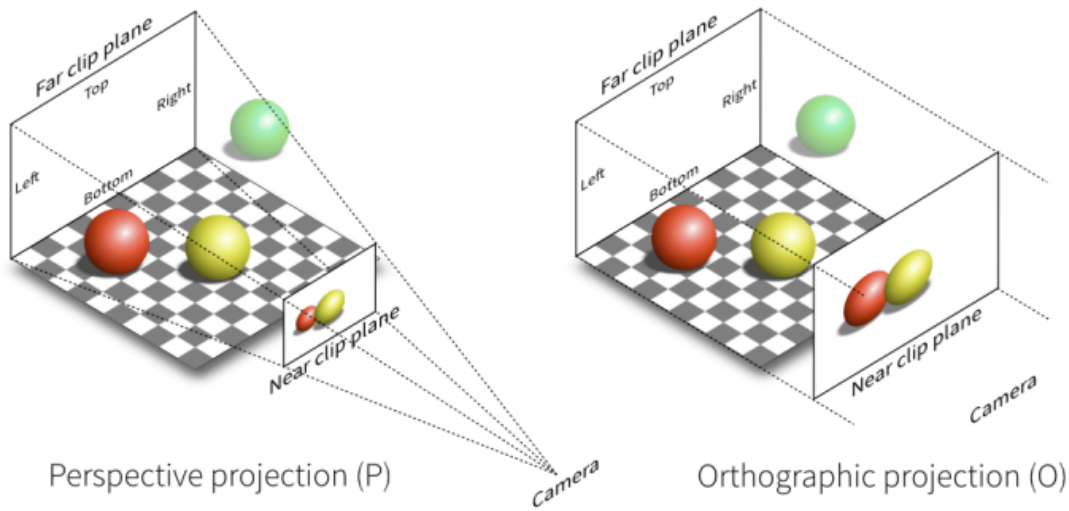
Animation / Simulation

- Key frame Animation
- Mass-spring System

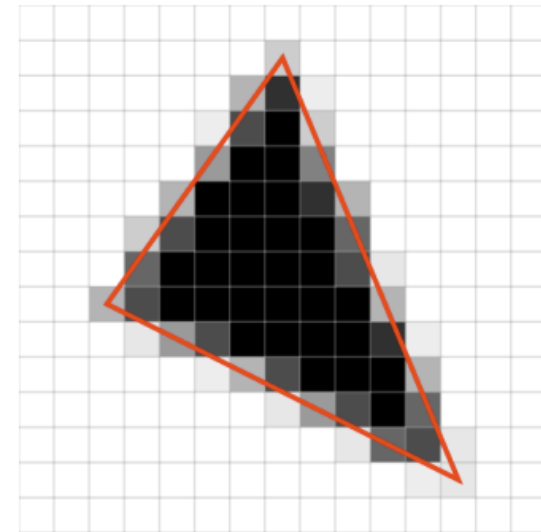


Rasterization

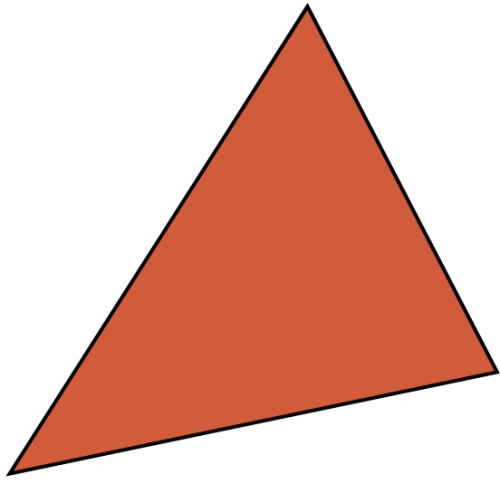
- Project **geometry primitives** (3D triangles / polygons) onto the screen
- Break projected primitives into **fragments** (pixels)
- Gold standard in Video Games (Real-time Applications)



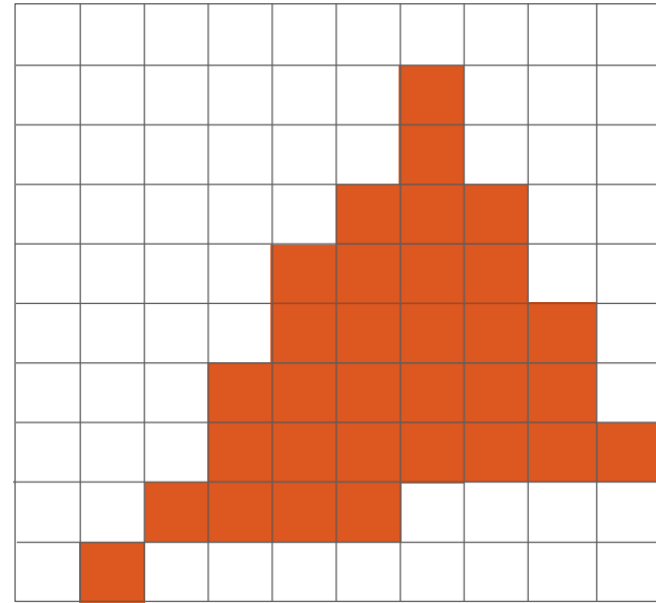
<http://vispy.org/modern-gl.html>



https://commons.wikimedia.org/wiki/File:Rasterisation-triangle_example.svg

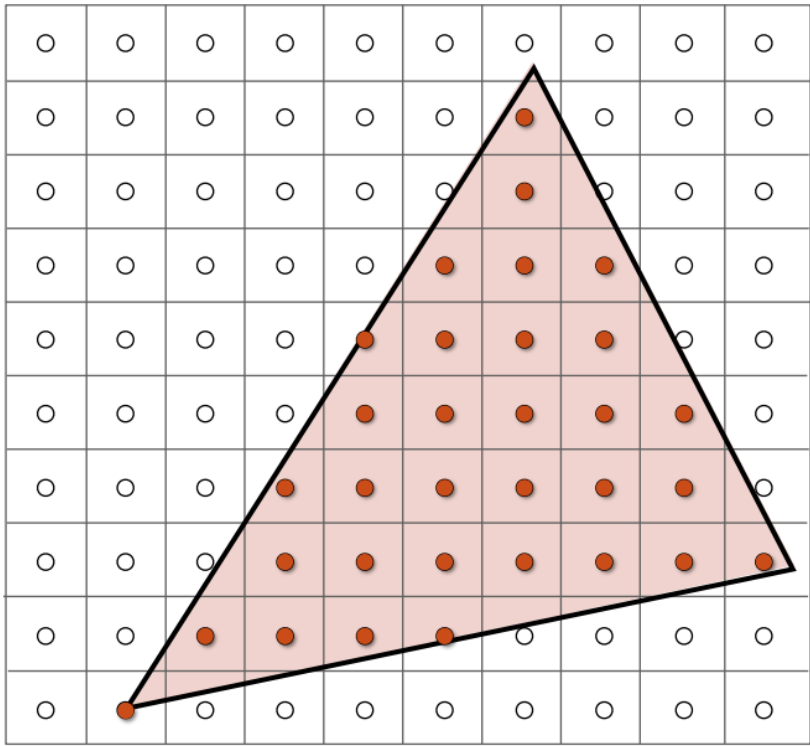


Continuous Triangle Function



After Rasterization

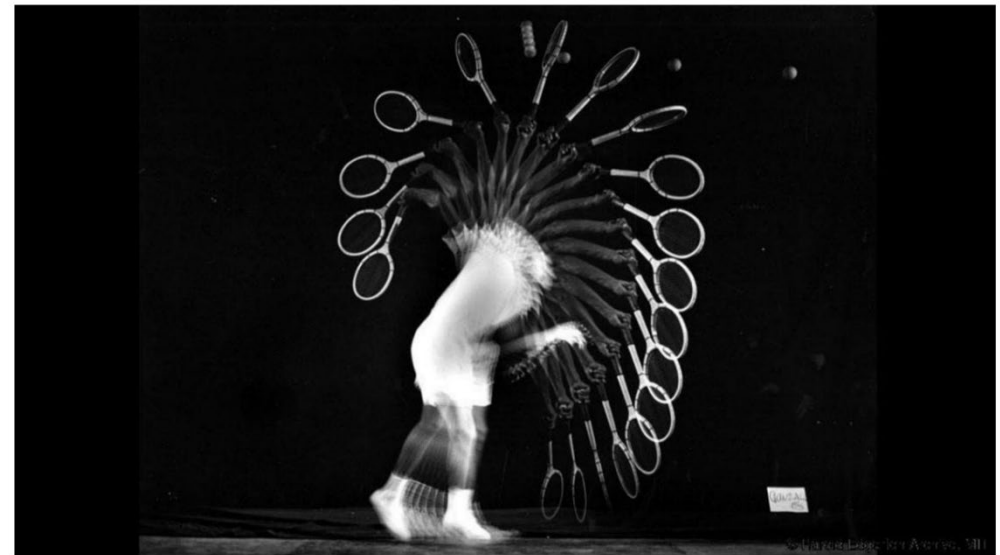
Jaggies! (Aliasing)



Rasterization = Sample 2D Positions

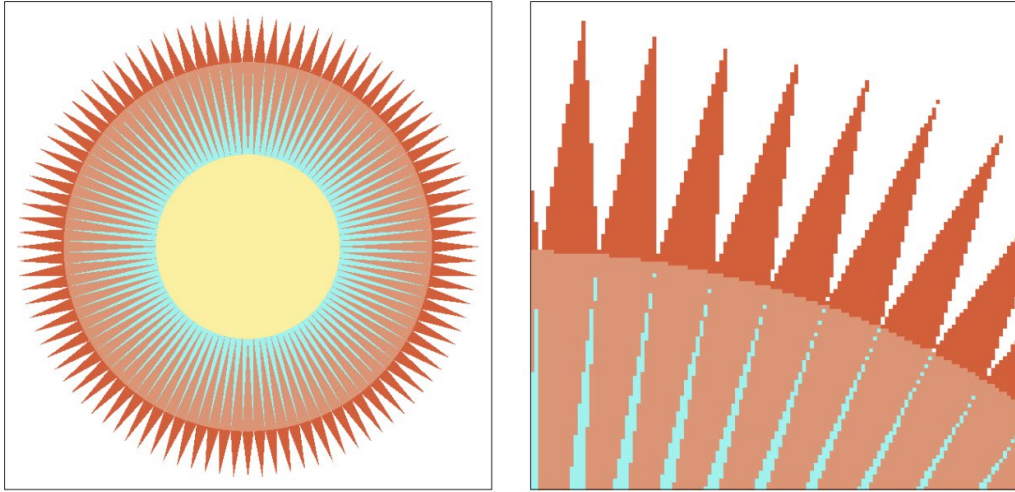


Photograph = Sample Image Sensor Plane



Video = Sample Time

Sampling Artifacts (Errors / Mistakes / Inaccuracies) in Computer Graphics



Jaggies (Aliasing)



Moiré Patterns



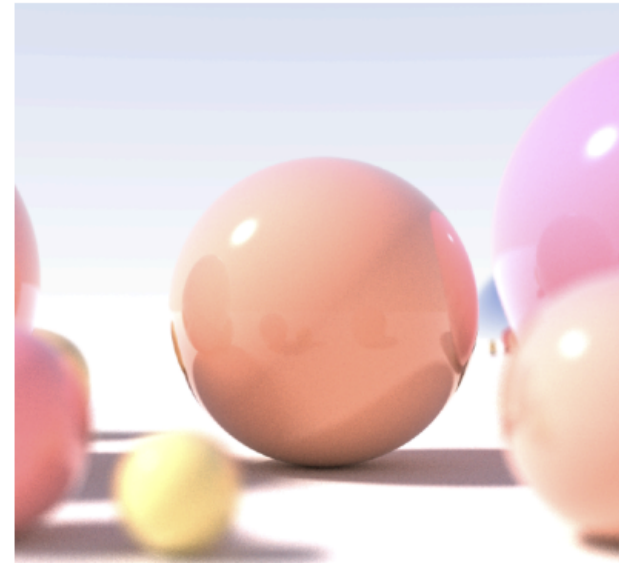
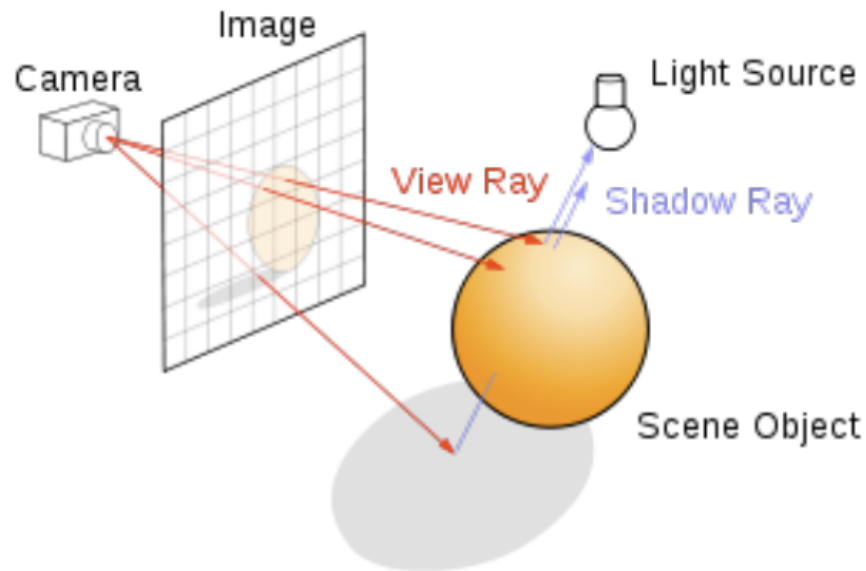
Wagon Wheel Illusion (False Motion)

How to do photo-realistic rendering?



Ray Tracing

- Shoot rays from the camera through each pixel
 - Calculate **intersection** and **shading**
 - **Continue to bounce** the rays till they hit light sources
- Gold standard in Animations / Movies (Offline Applications)



Radiometry

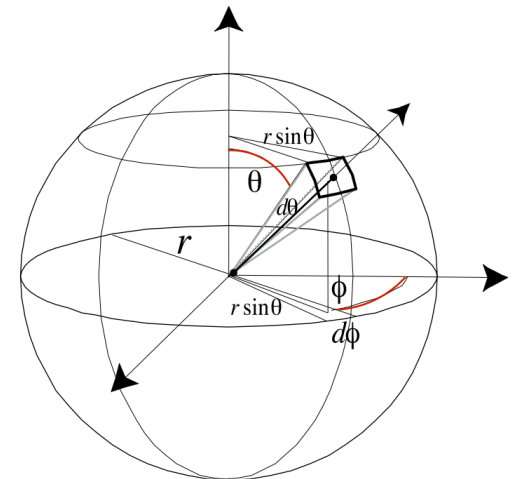
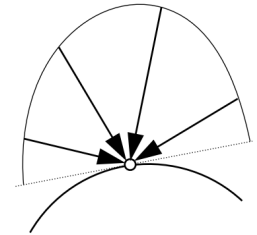
- Radiant flux (power):
 - the energy emitted, reflected, transmitted or received, per unit time

$$\Phi$$

- Irradiance:
 - How much light received by a “surface”
 - **Definition:** power per unit area on a surface point
 - **Lambert’s Law:** irradiance at surface is proportional to **cosine** of angle between light direction and surface normal.

$$E(x) = \frac{d\Phi}{dA}$$

where $dA = (rd\theta)(r \sin \theta d\phi) = r^2 \sin \theta d\theta d\phi$

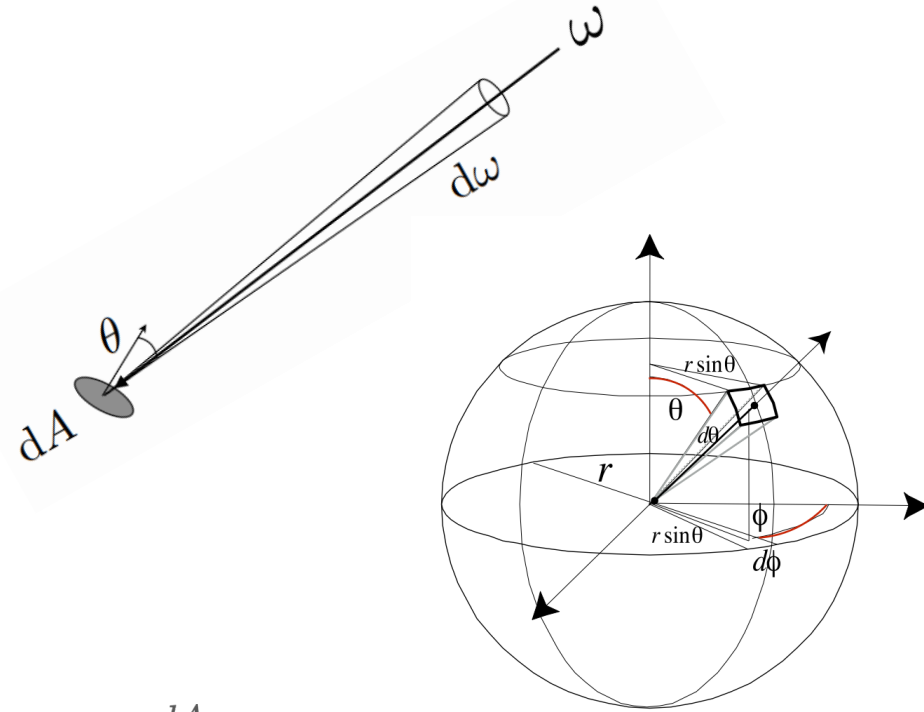


Radiometry

- Radiance:
 - How much light travelling along a “ray” (light received by an area from a direction)
 - **Definition:** power emitted, reflected, transmitted or received by a surface, per unit solid angle, per projected unit area

$$L(x, \omega) = \frac{d^2 \Phi}{d\omega dA \cos \theta}$$

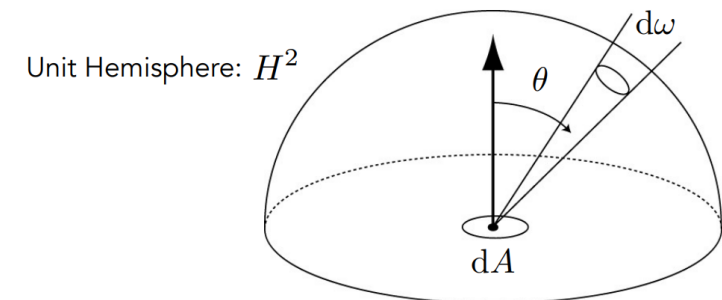
where $d\omega = \frac{dA}{r^2} = \sin \theta d\theta d\phi$



Radiance = Irradiance per solid angle

$$L(x, \omega) = \frac{dE(x)}{d\omega \cos \theta}$$

$$E(x) = \int_{H^2} L(x, \omega) \cos \theta d\omega$$



BRDF

Bidirectional Reflectance Distribution Function (BRDF)

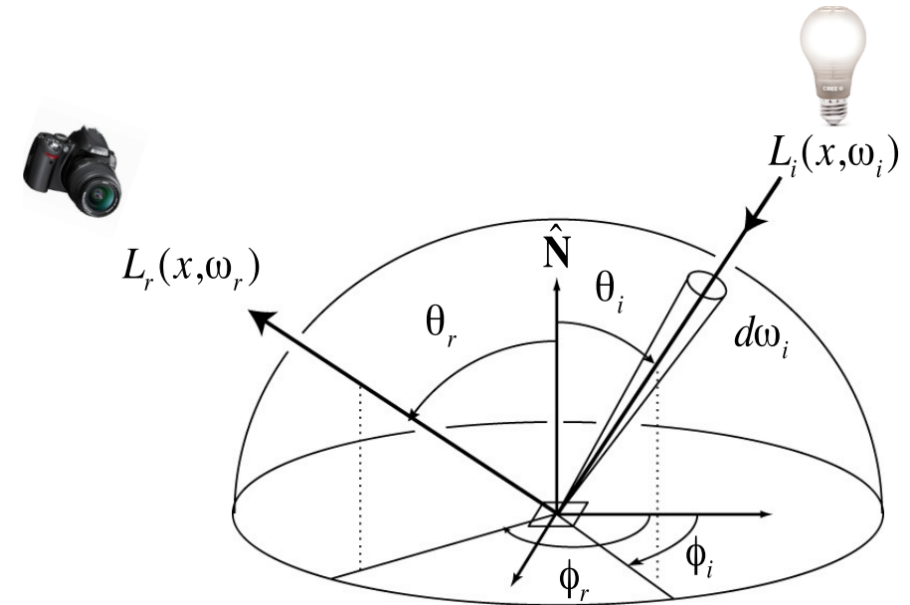
- How much light is reflected into each outgoing direction ω_r from each incoming direction ω_i
- BRDF can be simply regarded as diffusion + specular (we will discuss this in detail in later classes).

$$f_{BRDF}(\omega_i \rightarrow \omega_r) = \frac{dL_r(\omega_r)}{\underbrace{dE_i(\omega_i)}_{\text{irradiance}}} = \frac{dL_r(\omega_r)}{L_i(\omega_i) \cos \theta_i dw_i}$$

The Reflection Equation:

- Total light reflected from the outgoing direction ω_r

$$\underbrace{L_r(x, \omega_r)}_{\text{Outgoing radiance}} = \int_{H^2} f_{BRDF}(x, \omega_i \rightarrow \omega_r) dE_i(x, \omega_i)$$
$$= \int_{H^2} f_{BRDF}(x, \omega_i \rightarrow \omega_r) \underbrace{L_i(x, \omega_i)}_{\text{Incoming radiance}} \cos \theta_i dw_i$$



The Rendering Equation

- The rendering equation can be derived by adding an **emission term** to the reflection equation.

$$L_r(x, \omega_r) = L_e(x, \omega_r) + \int_{H^2} L_i(x, \omega_i) f_{BRDF}(x, \omega_i \rightarrow \omega_r) (n \cdot \omega_i) d\omega_i$$

Reflected light
(output image)

Emission

Incident light

BRDF

Cosine of
incident angle

- The rendering equation is a Fredholm Integral Equation of second kind:

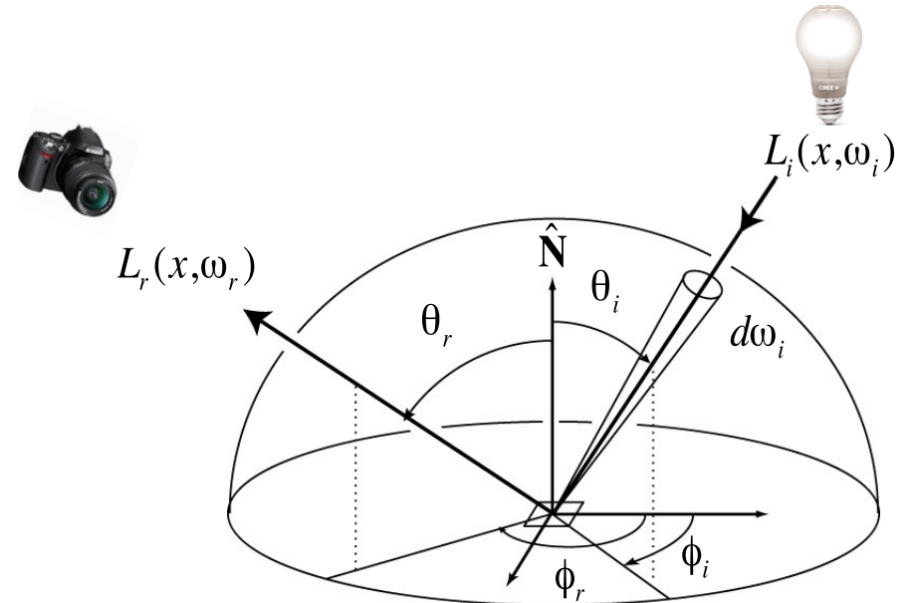
$$l(u) = l_e(u) + \int l(v) k(u, v) dv$$

- Use linear operators:

$$L = L_e + \textcircled{K} L$$

Light transport matrix

$$(K \circ f)(x) = \int k(x, x') f(x') dx$$



The Rendering Equation

- The rendering equation can be derived by adding an **emission term** to the reflection equation.

$$L_r(x, \omega_r) = L_e(x, \omega_r) + \int_{H^2} L_i(x, \omega_i) f_{BRDF}(x, \omega_i \rightarrow \omega_r) (n \cdot \omega_i) d\omega_i$$

Reflected light
(output image)

Emission

Incident light

BRDF

Cosine of
incident angle

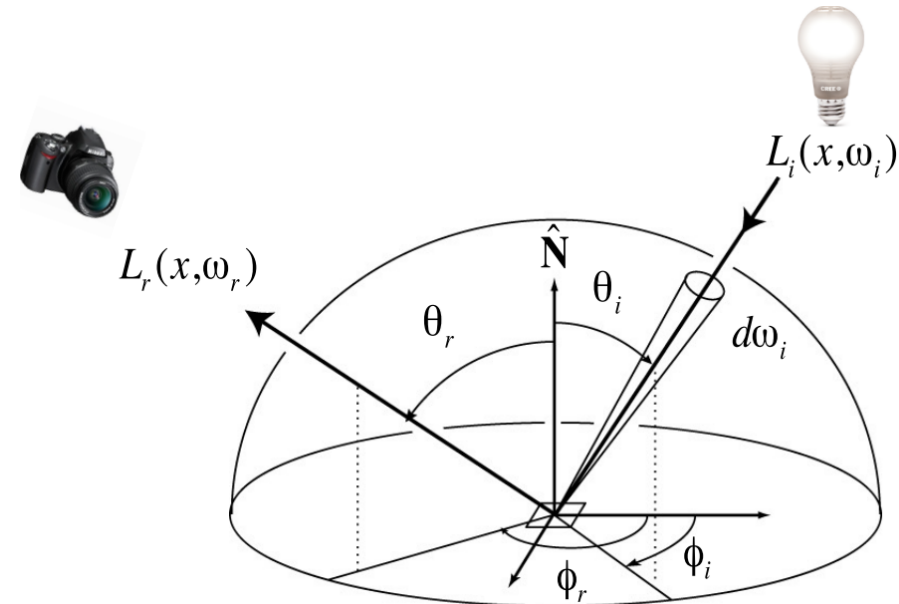
- The rendering equation is a Fredholm Integral Equation of second kind:

$$l(u) = l_e(u) + \int l(v) k(u, v) dv$$

- Use linear operators:

$$L = L_e + \underbrace{KL}_{\text{Light transport matrix}}$$

$$(K \circ f)(x) = \int k(x, x') f(x') dx$$



Ray Tracing

Solve the rendering equation using linear operators

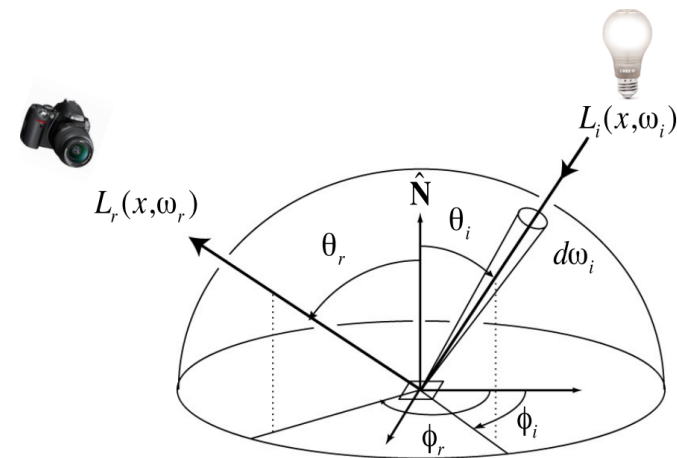
$$L = L_e + KL$$

$$\begin{aligned} L &= (I - K)^{-1} L_e \\ &= (I + K + K^2 + K^3 + \dots) L_e \\ &= L_e + KL_e + K^2 L_e + K^3 L_e + \dots \end{aligned}$$

Emission
directly from
light sources

It involves:

- (1) solving the integral over the hemisphere
- (2) **recursive execution.**



L_e



$K \circ L_e$



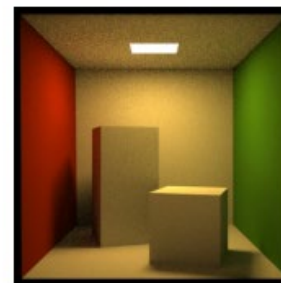
$K \circ K \circ L_e$



$K \circ K \circ K \circ L_e$



$L_e + K \circ L_e$



$L_e + \dots K^2 \circ L_e$

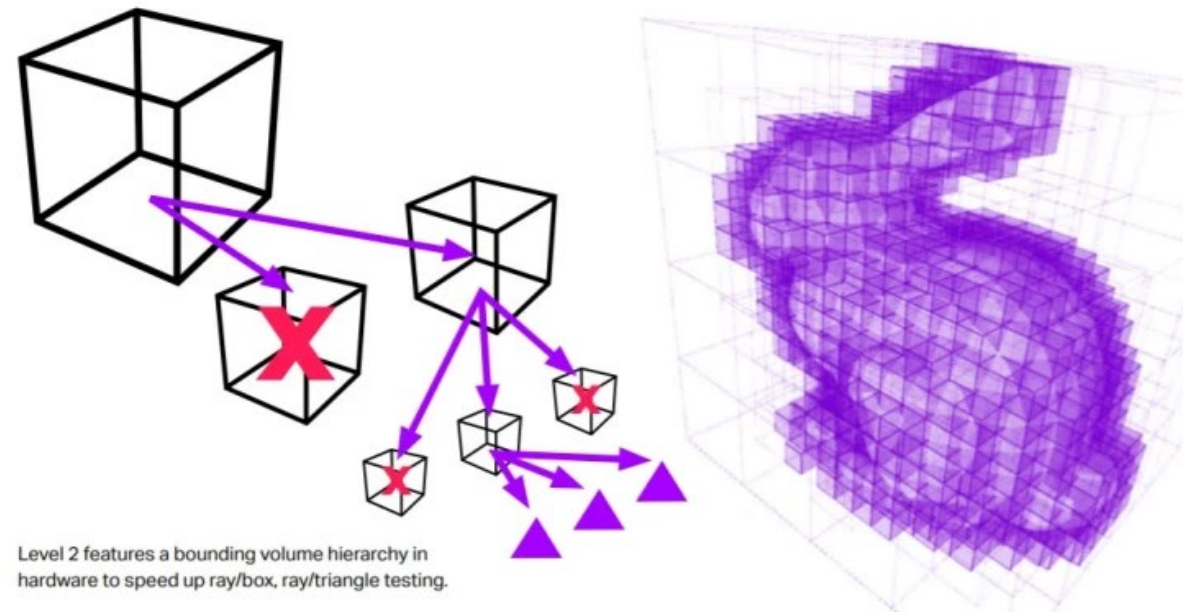


$L_e + \dots K^3 \circ L_e$

Towards real-time ray tracing

Real-time ray tracing itself is an active research area. Several aspects can be used to improve the rendering efficiency:

- **Efficient data structure (e.g. bounding volume hierarchy (BVH))**
 - Use a tree structure to partition the space which speed-up the ray intersection.
- Importance Sampling
- Learning based denoising

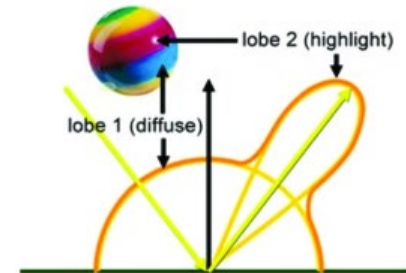


Towards real-time ray tracing

Real-time ray tracing itself is an active research area. Several aspects can be used to improve the efficiency:

- Efficient data structure (e.g. bounding volume hierarchy (BVH))
- **Importance Sampling on Materials (BRDF)**
 - Instead of uniformly sampling on the semi-sphere, we can sample more points based on the shape of BRDF (diffusion + specular).
- Learning based denoising

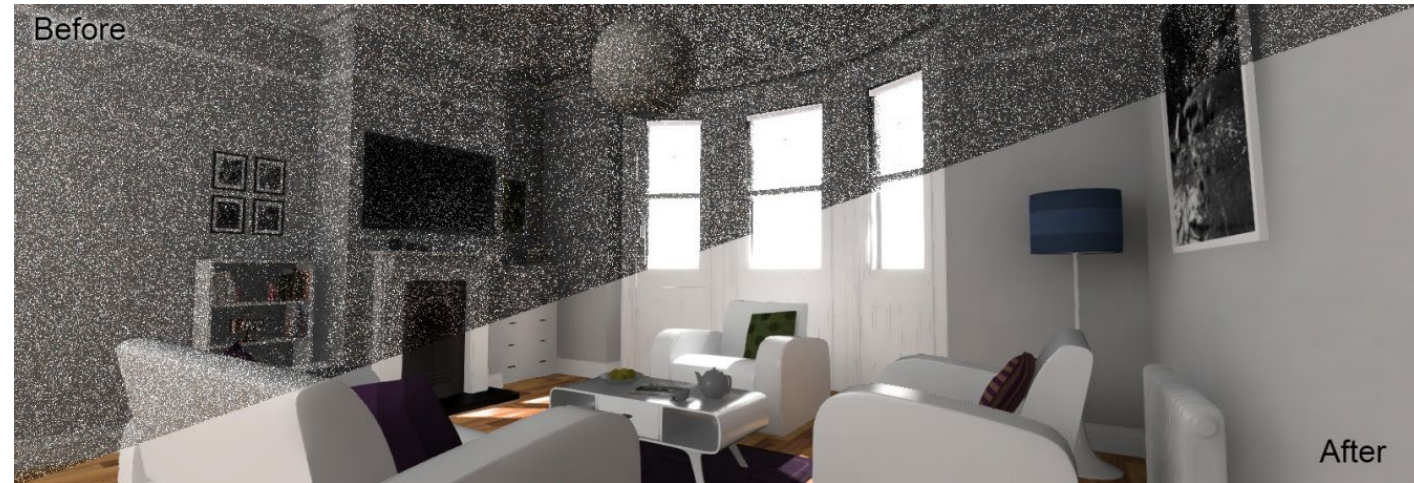
materials: sample important "lobes"



Towards real-time ray tracing

Real-time ray tracing itself is an active research area. Several aspects can be used to improve the efficiency:

- Efficient data structure (e.g. bounding volume hierarchy (BVH))
- Importance Sampling
- **Learning based denoising/super-sampling**
 - Use deep learning techniques, it is also possible to trace less paths for each pixel or lower resolution image, and then use neural network to get the final image.



Results of ray tracing: Photo-realistic



Other Modern Ray Tracing Algorithms

- Bidirectional path tracing
- Photon Mapping
- Metropolis light transport
- Multiple Importance Sampling (MIS)
- Quasi-Monte Carlo methods (QMC)
- Finite Element Radiosity
- ...

Overview of This Class

0. Neural Fields

I. Fundamentals of Classical Rendering Techniques in Computer Graphics

II. Two pioneering works in Neural Scene Representations and Neural Rendering

- Scene Representation Networks (SRN)
- Neural Volumes (before that: Deep Appearance Models)

Scene Representation Networks (SRN)

Sitzmann, Zollhoefer, Wetzstein

NeurIPS 2019



Infer Neural Scene Representation from 2D observations.



Neural Scene
Representation

Learned feature
representation of
scene.

Formulate Neural Renderer.



Neural Scene
Representation

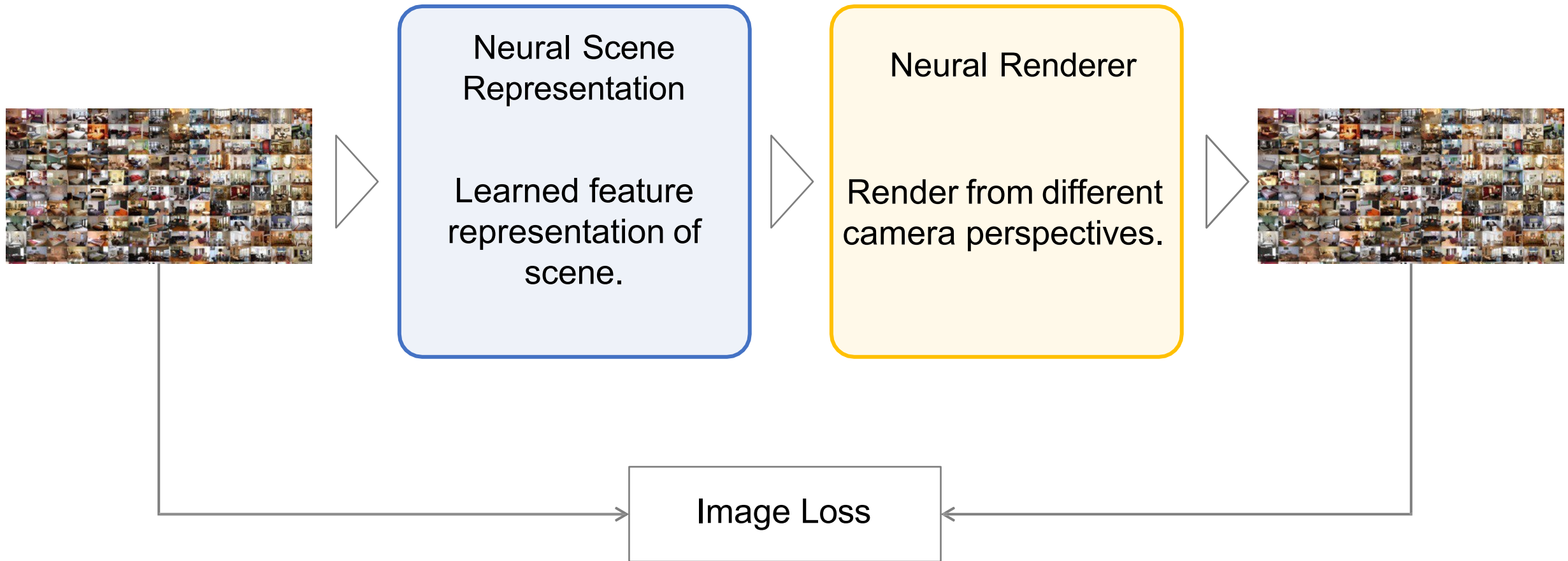
Learned feature
representation of
scene.



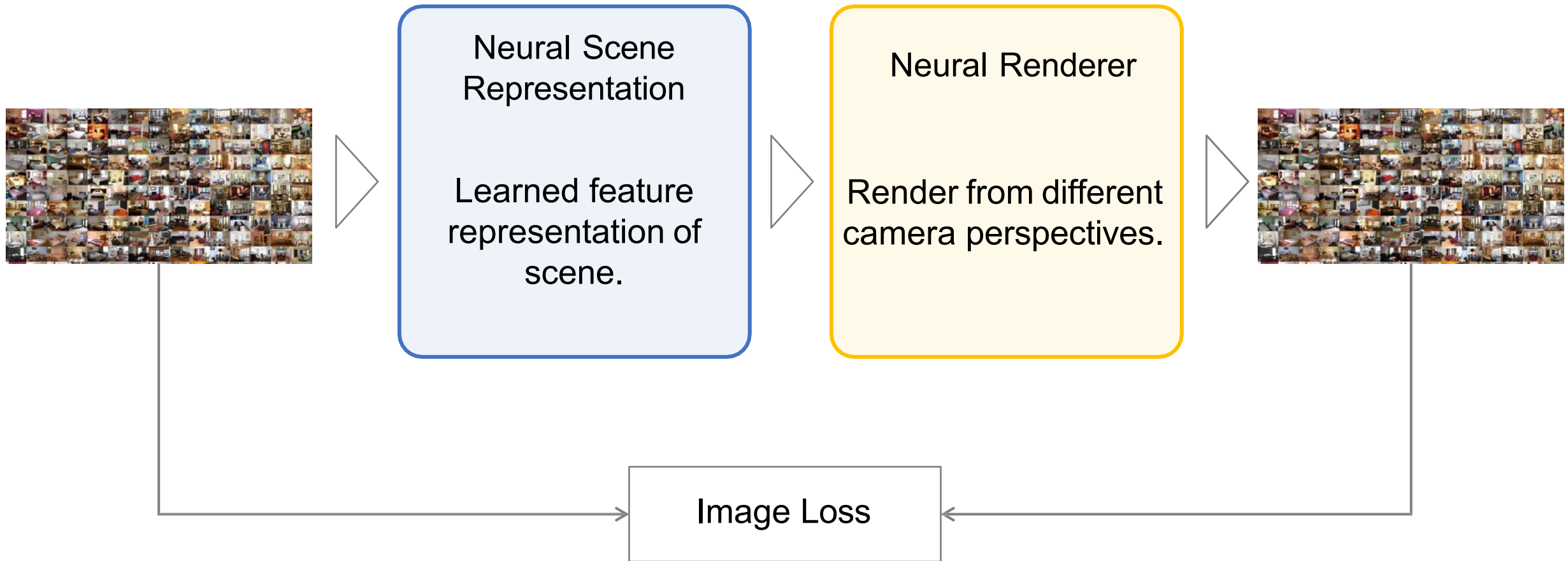
Neural Renderer

Render from different
camera perspectives.

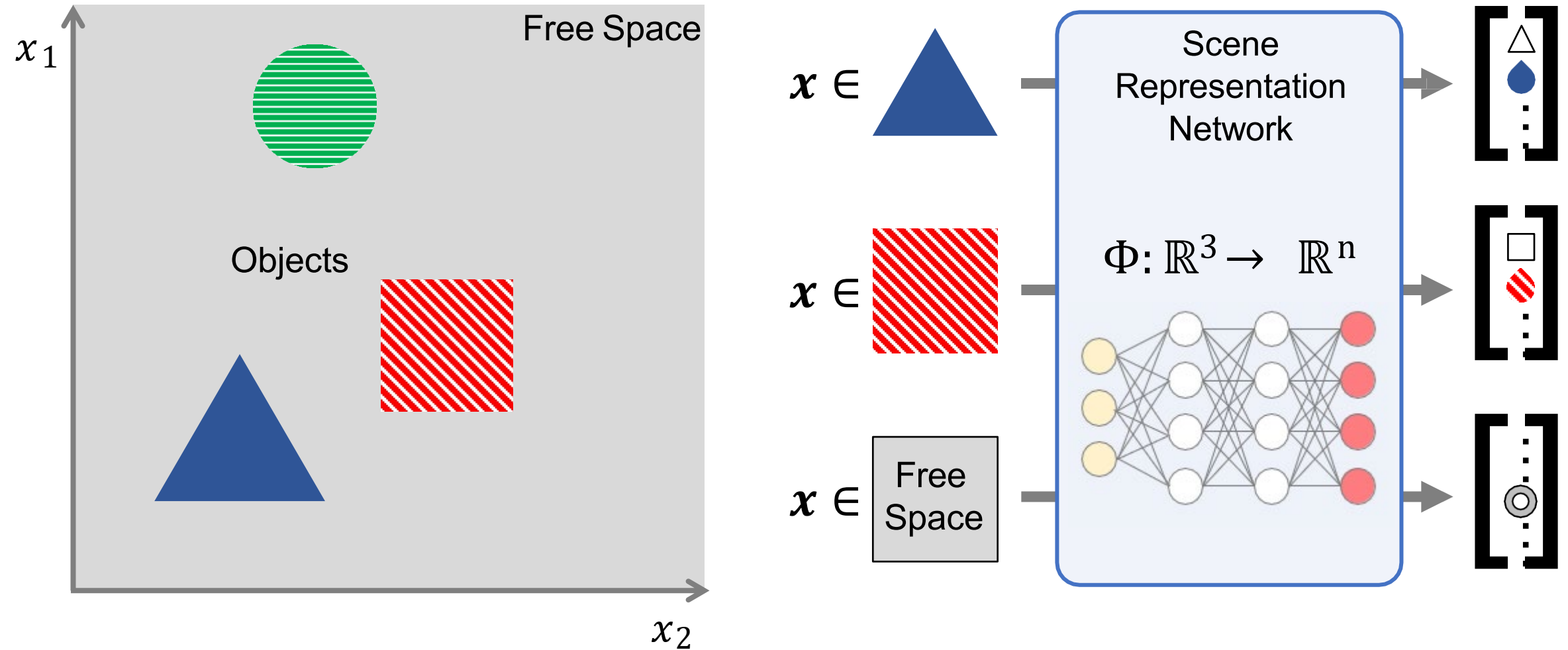
Finally: Predict training views & enforce loss on re-rendering error!



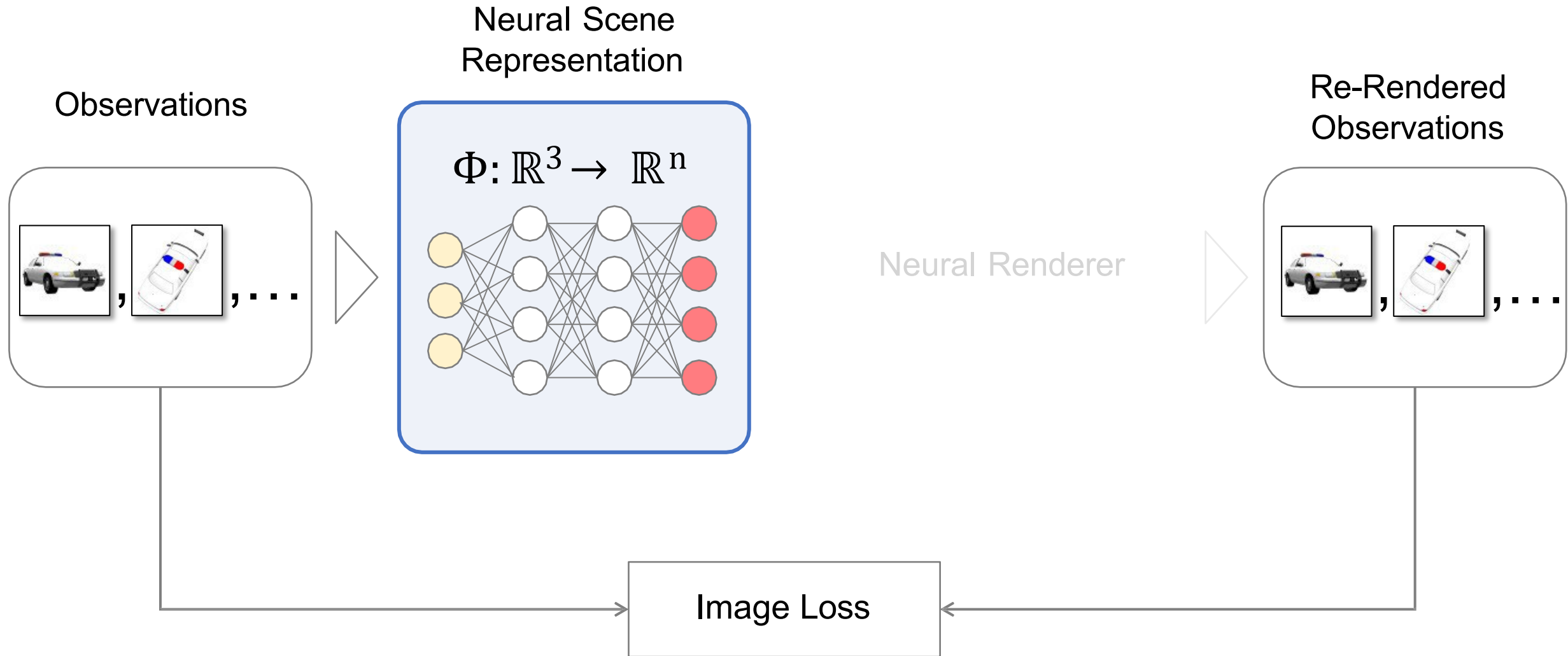
Self-supervised Scene Representation Learning



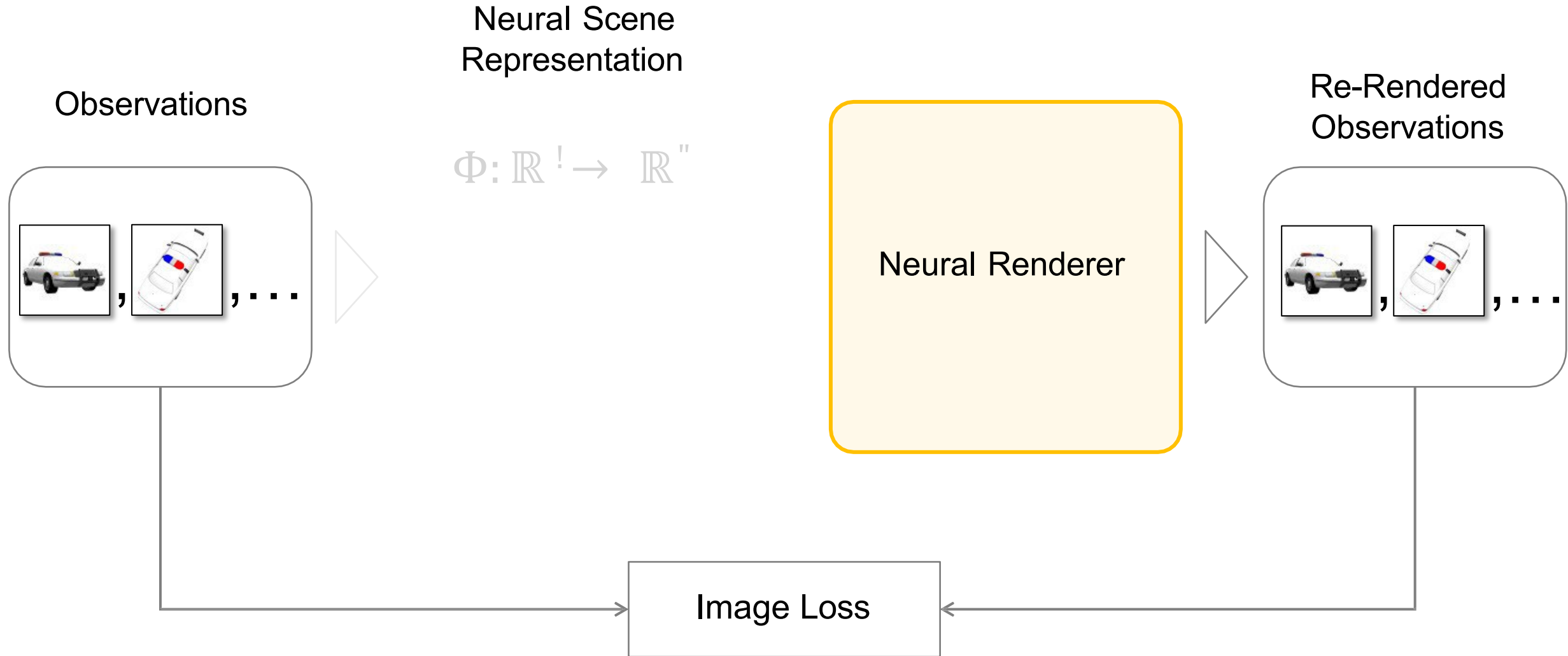
Scene Representation Network parameterizes scene as MLP.



Scene Representation Networks

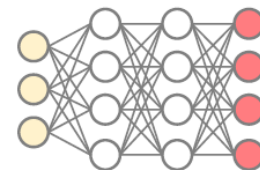


Scene Representation Networks

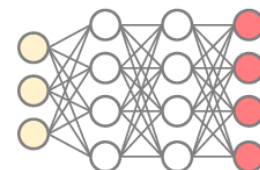


Each scene represented by its own SRN.

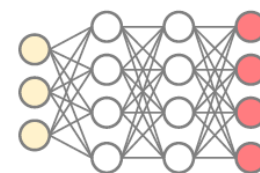
parameters $\phi_0 \in \mathbb{R}^l$



parameters $\phi_1 \in \mathbb{R}^l$

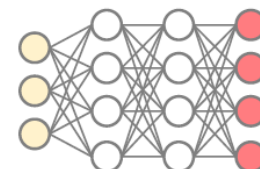


parameters $\phi_2 \in \mathbb{R}^l$



○ ○ ○

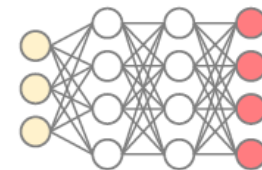
parameters $\phi_n \in \mathbb{R}^l$



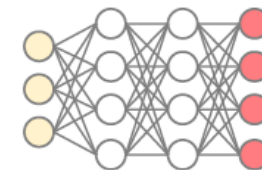
Manifold assumption.

ϕ_i live on k -dimensional subspace of $\mathbb{R}^l$, $k < l$.

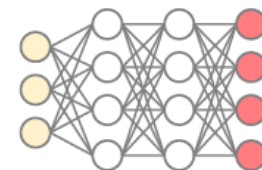
parameters $\phi_0 \in \mathbb{R}^l$



parameters $\phi_1 \in \mathbb{R}^l$

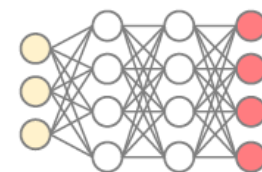


parameters $\phi_2 \in \mathbb{R}^l$

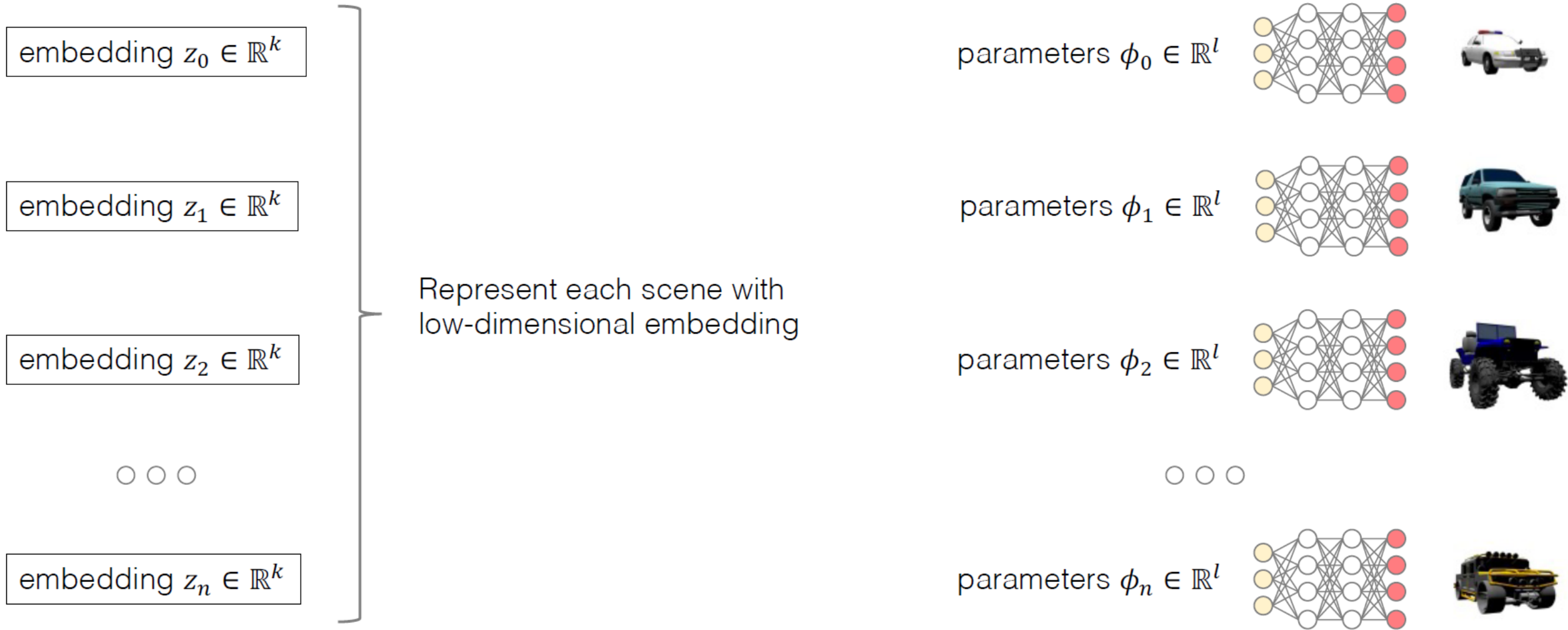


○ ○ ○

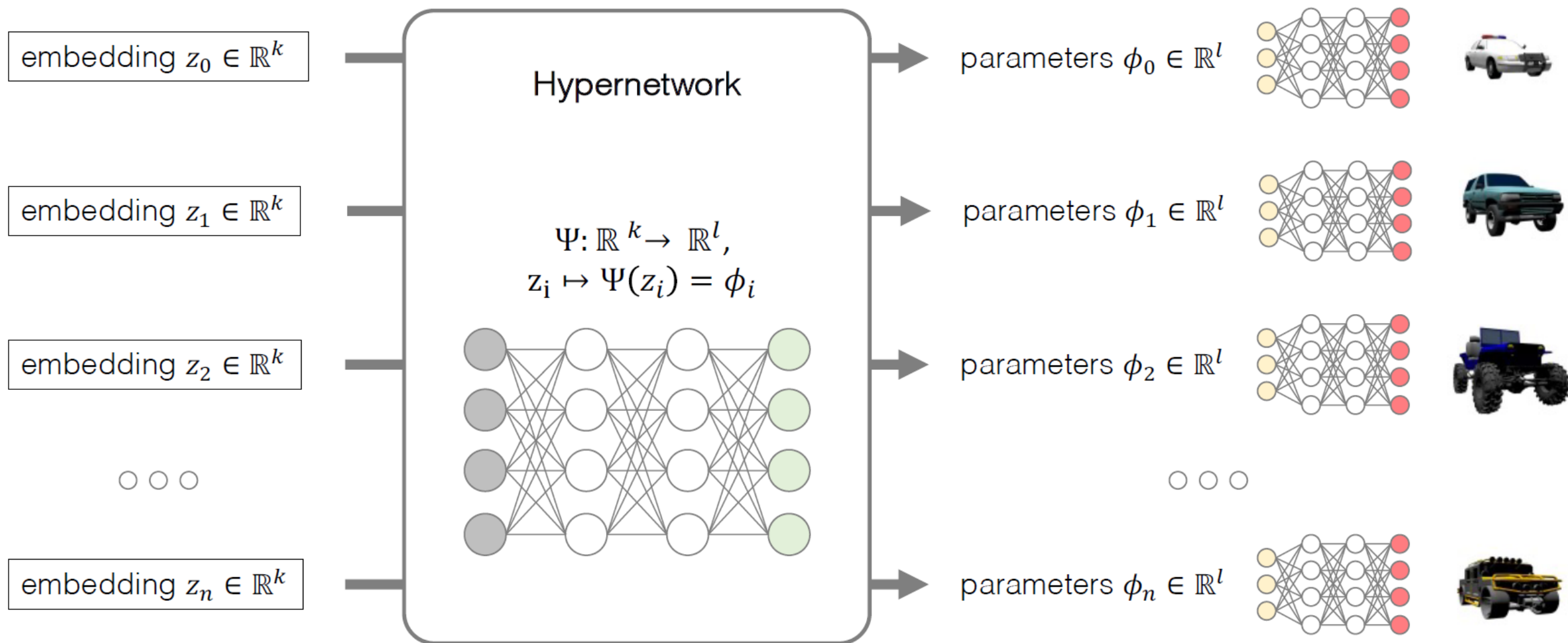
parameters $\phi_n \in \mathbb{R}^l$



Represent each scene by low-dimensional embedding.



Map embeddings to SRN parameters via Hypernetwork.



Novel View Synthesis – Baseline Comparison

Shapenet v2 cars – training set objects

Tatarchenko et al.



Worrall et al.



Deterministic
GQN



SRNs



Training on:

- 2434 cars
- 50 observations each

Testing on:

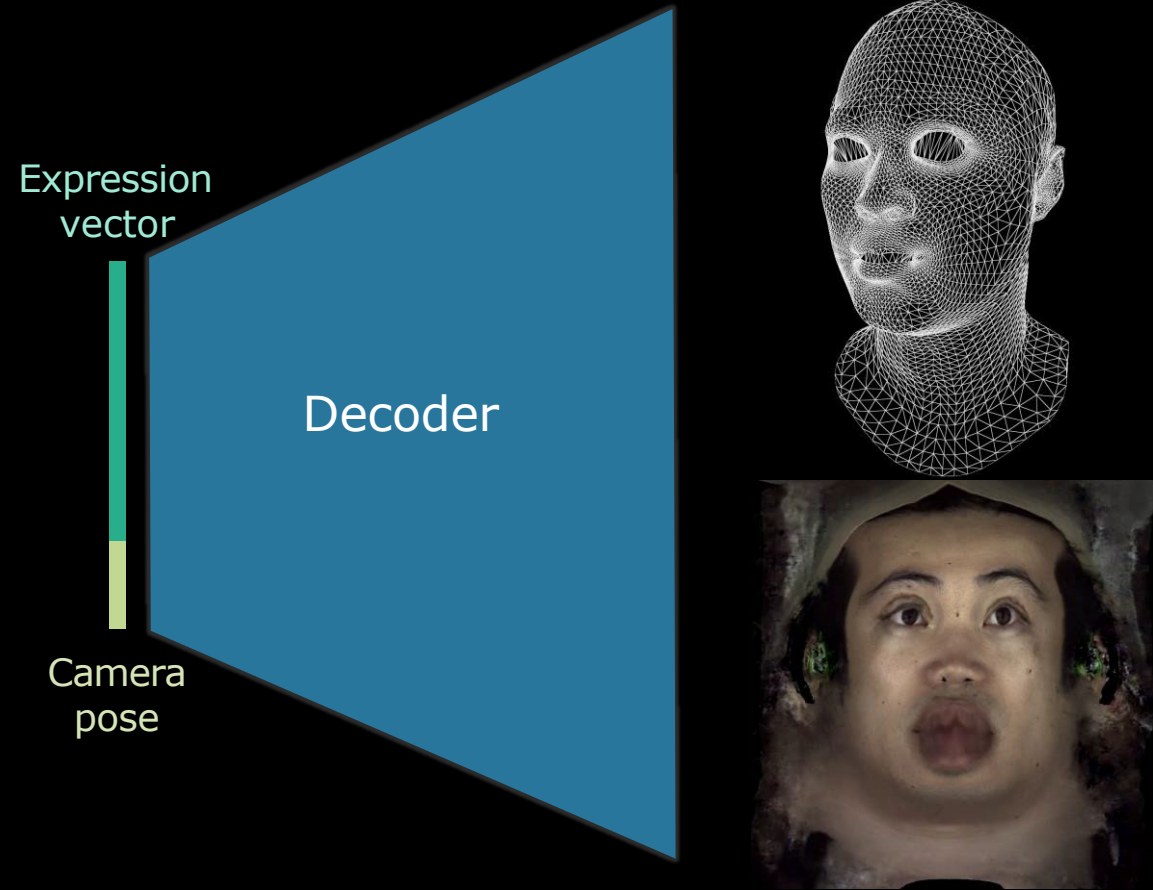
- 2434 cars from training set
- 250 novel views rendered in Archimedean spiral around each object

Neural Volumes

Lombardi, Simon, Saragih, Schwartz, Lehrmann, Sheikh
SIGGRAPH 2019

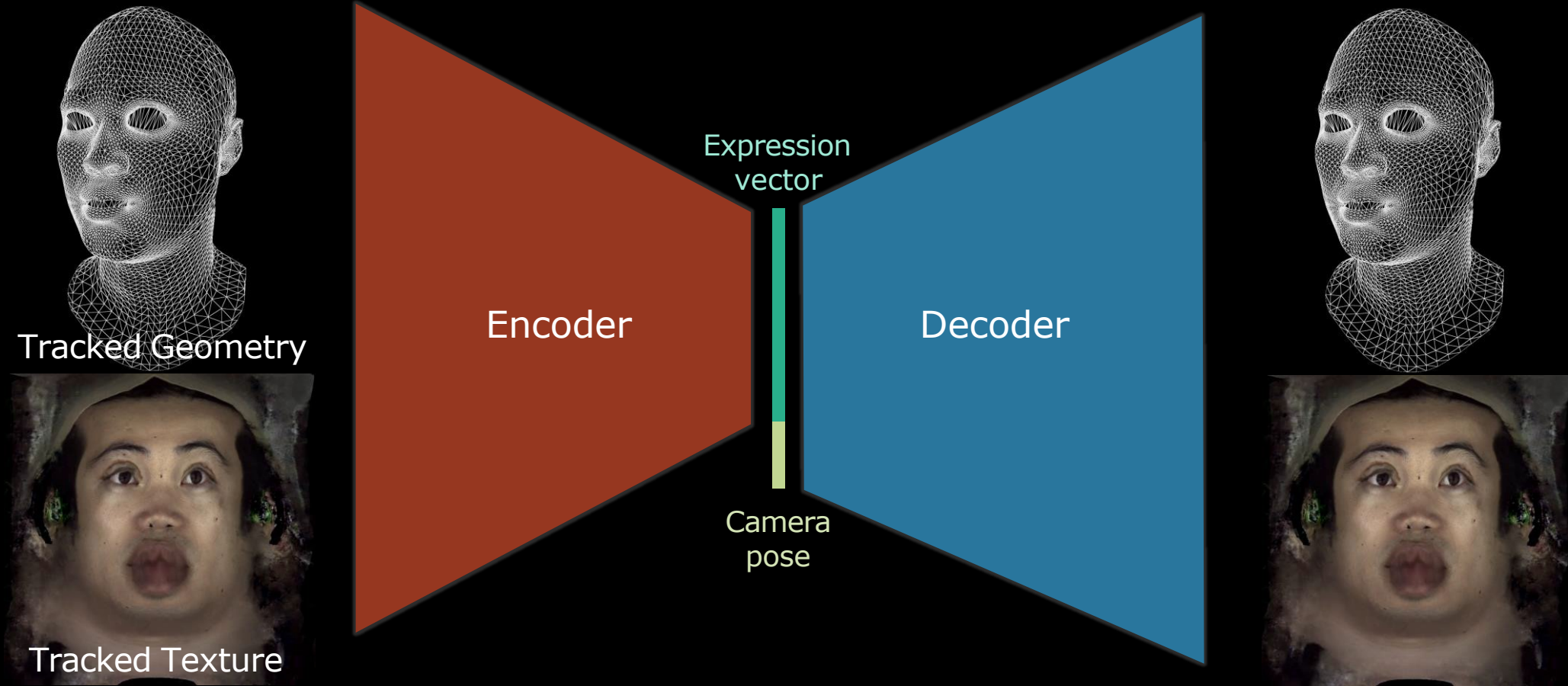
Deep Appearance Models

View-Conditioned Decoder

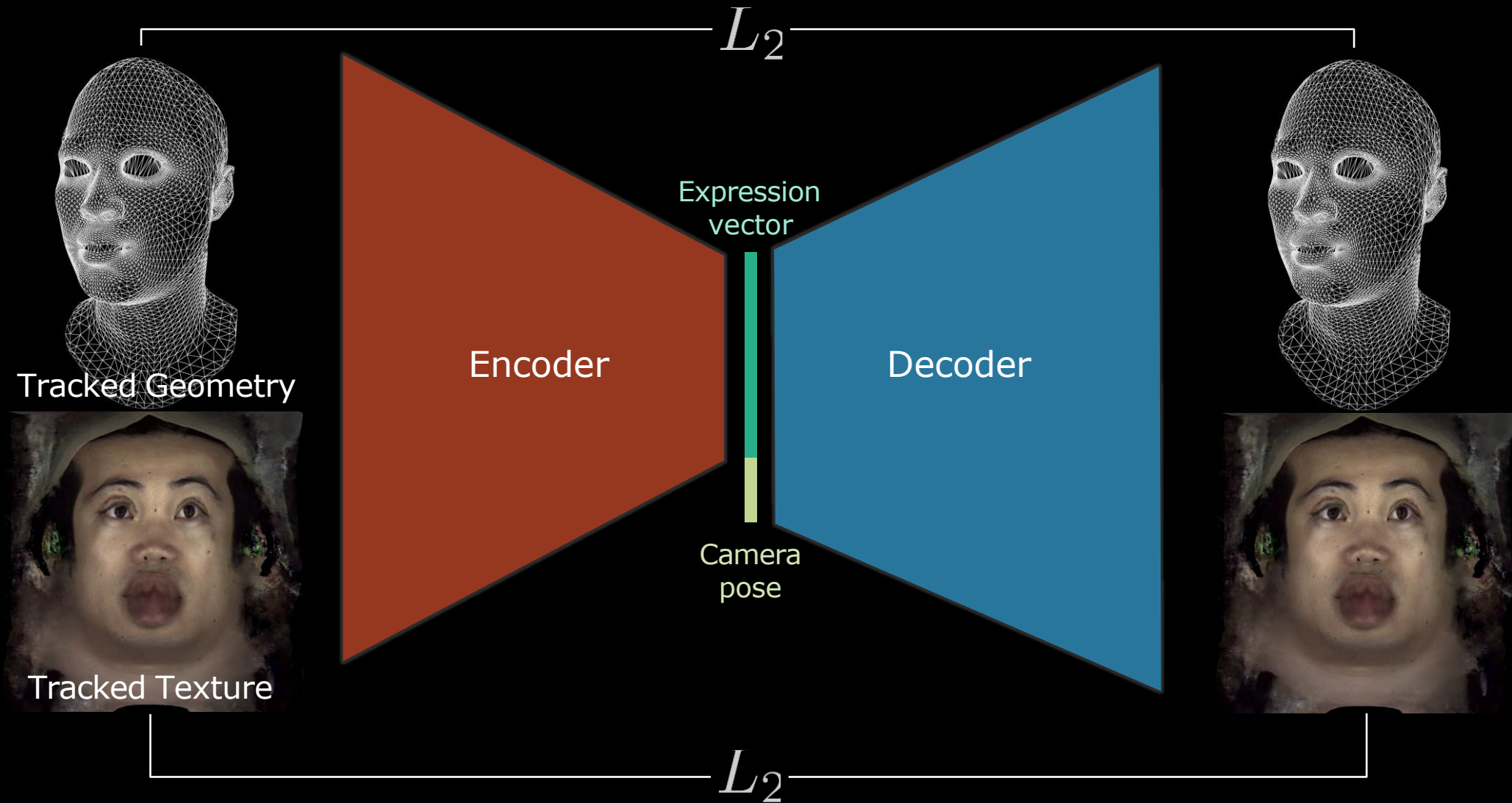


Lombardi, Saragih, Simon, Sheikh
SIGGRAPH 2018

Deep Appearance Models



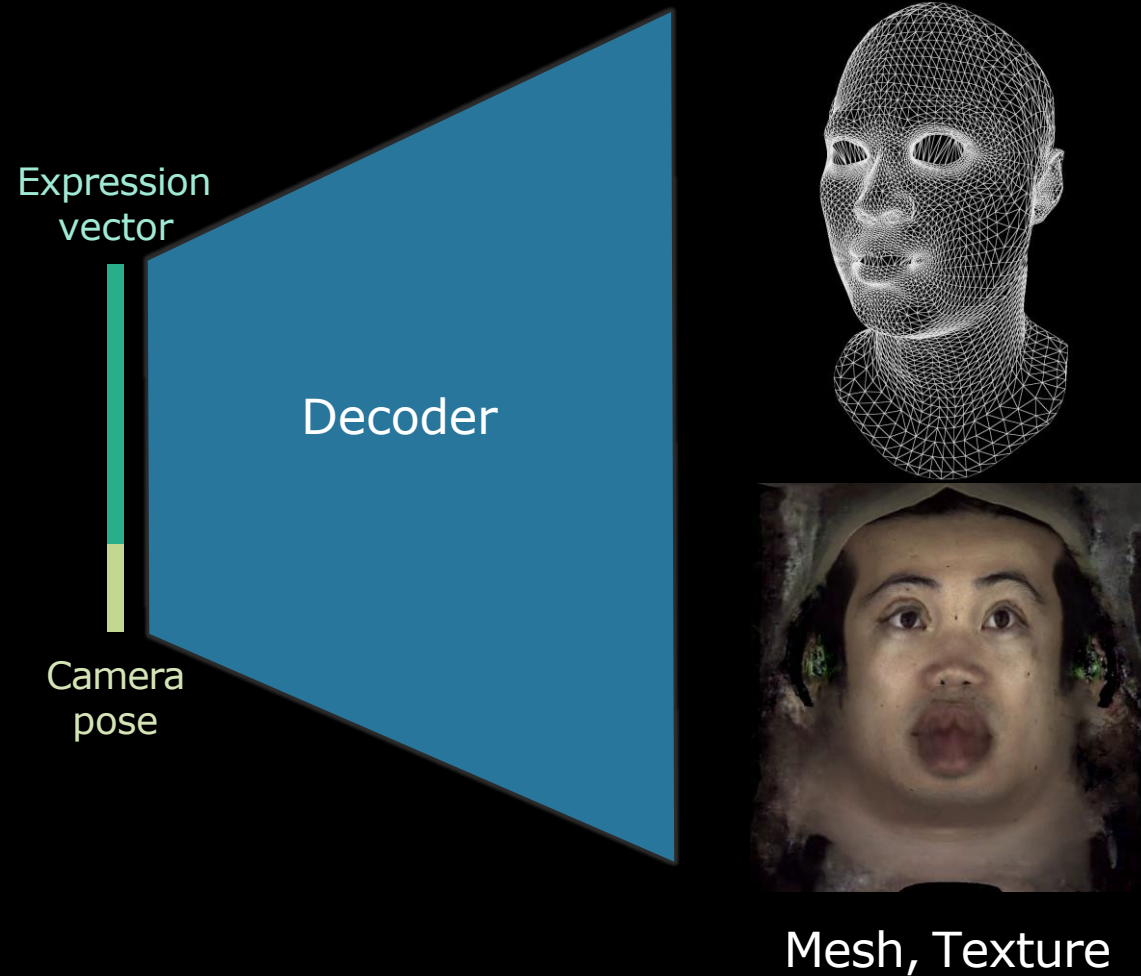
Deep Appearance Models



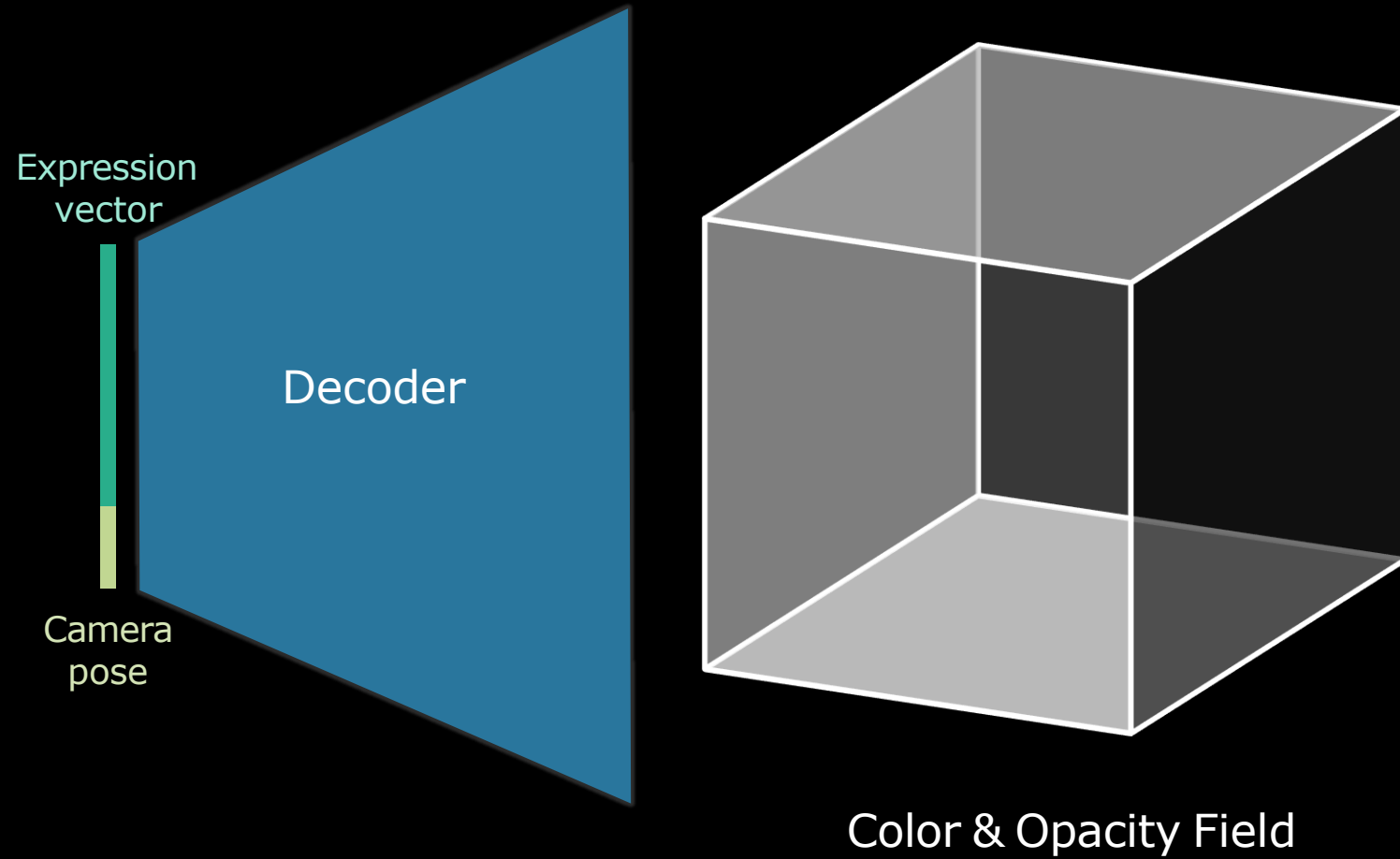




Mesh/Texture Decoder

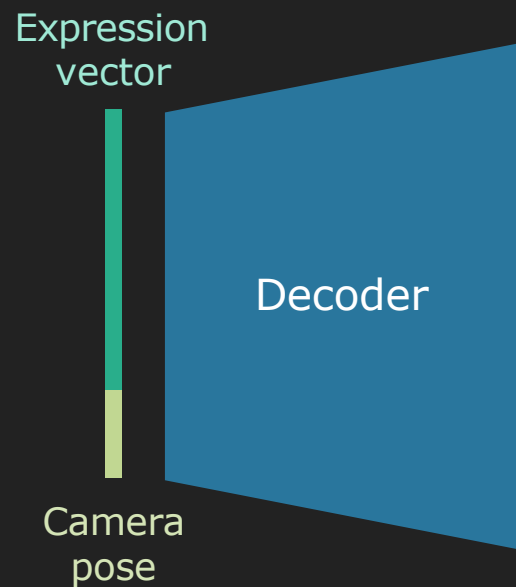


Volume Decoder

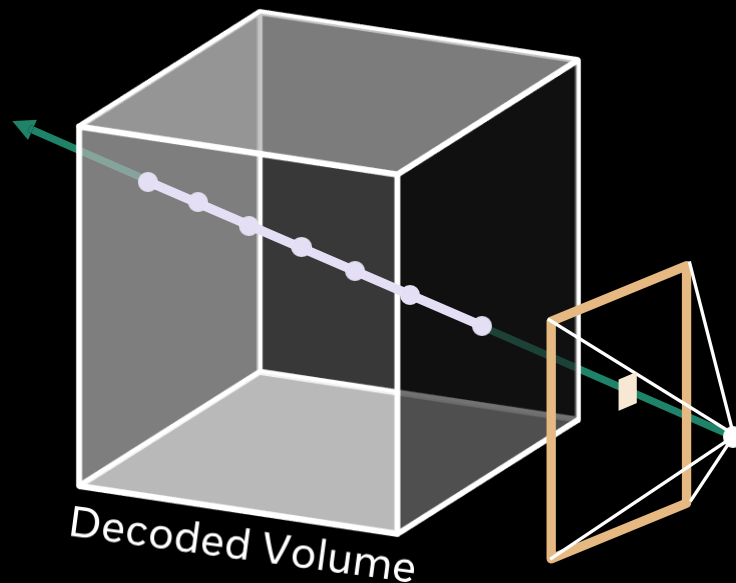


Volumetric Neural Rendering

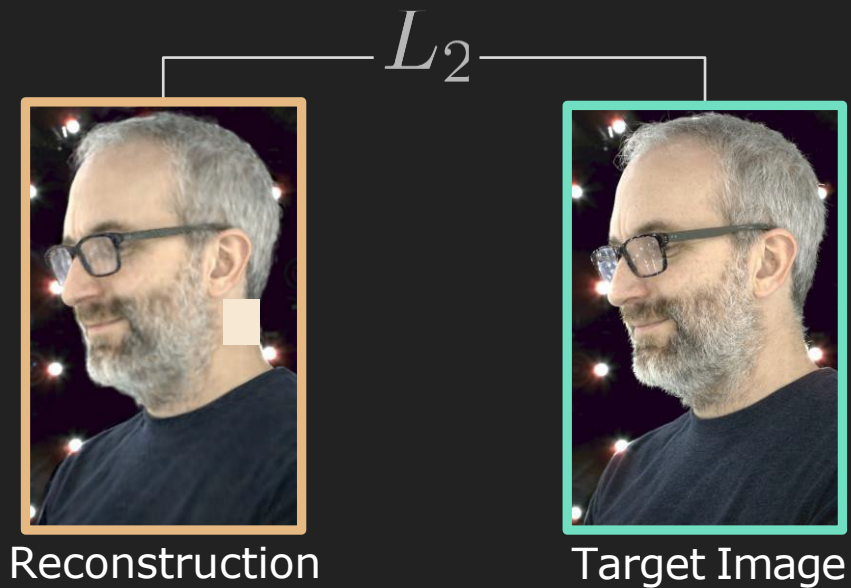
Decoding



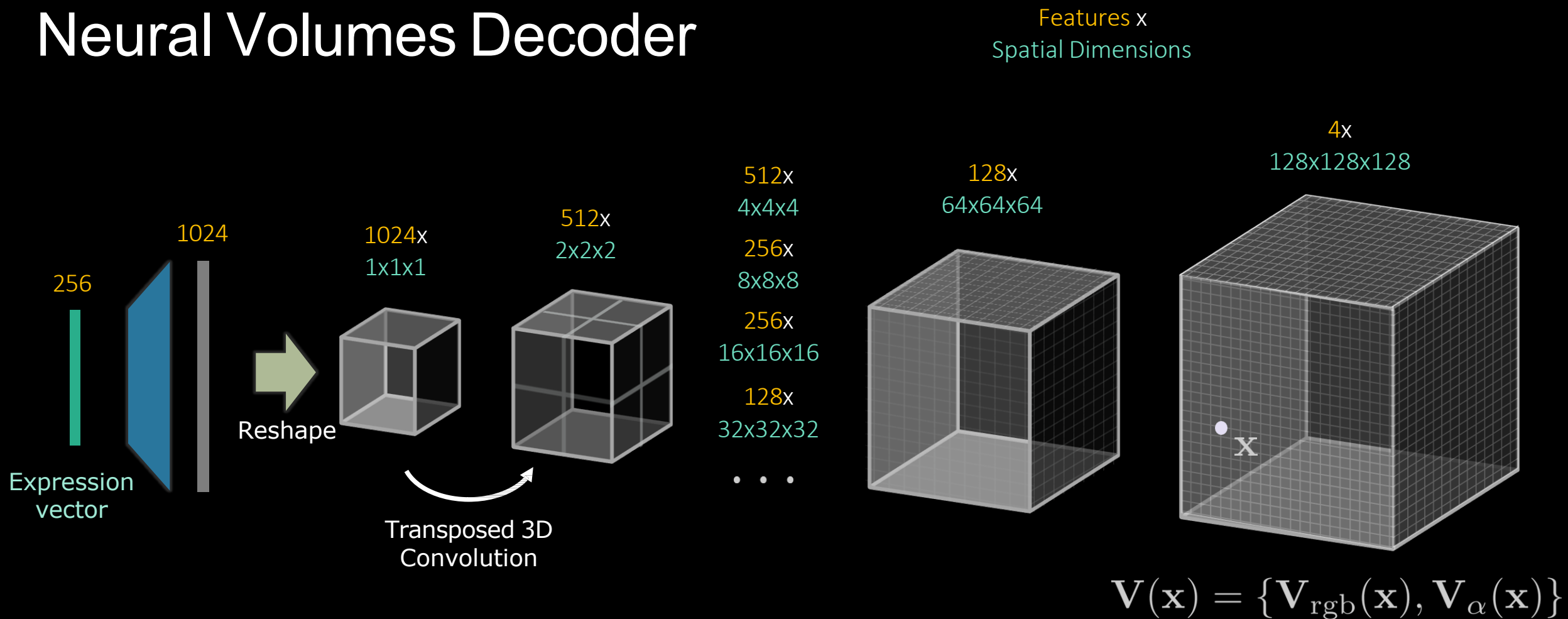
Raymarching



Training

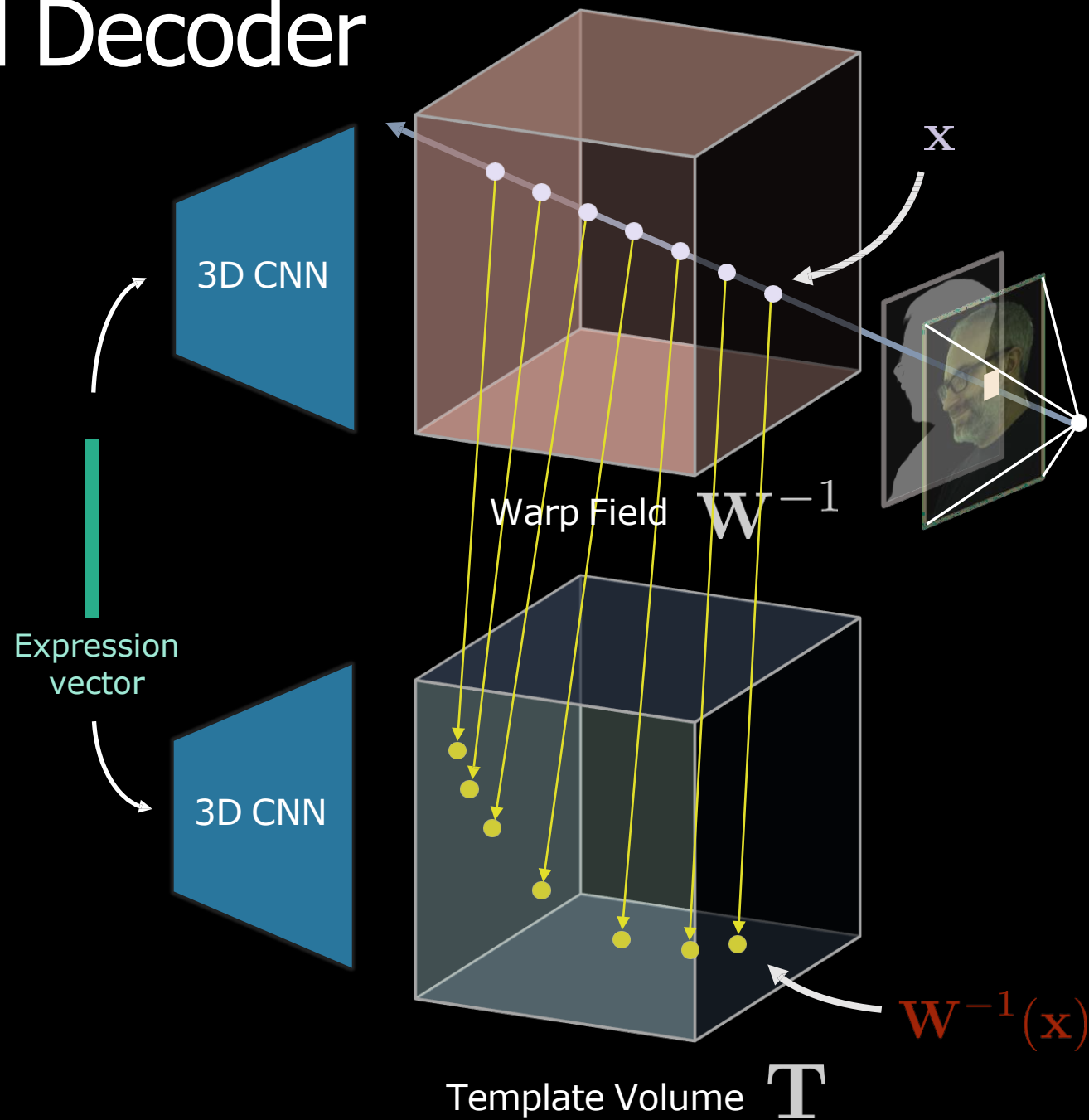


Neural Volumes Decoder





Warp Field Decoder





Volume with Warping



Template

Example Reconstructions



Acknowledgments

- Advances in Neural Rendering
- Neural Fields in Visual Computing and Beyond
- awesome-NeRF: a curated list of awesome neural radiance fields papers
- MPII Summer Semester 2023: Computer Vision and Machine Learning for Computer Graphics
- Vincent Sitzmann's Slides for SRN, Stephen Lombardi's Slides for Neural Volumes, Ben Mildenhall's slides for NeRF, Lingqi Yan's Slides for Rendering

Any Questions?